

KatanaKIT CSS — Documentación

Un mini-framework SCSS ligero y modular: tokens de diseño, clases de utilidad y mixins de layout sin sobrecarga en tiempo de ejecución.

Índice

Empieza aquí.....	3
KatanaKIT CSS.....	3
Primeros pasos.....	4
Conceptos básicos.....	8
Tokens de diseño.....	8
Colores.....	12
Breakpoints.....	15
Modo oscuro.....	19
@apply.....	22
Utilidades.....	26
Ancho de borde.....	26
Display.....	27
Direccion flex.....	29
Tamano de fuente.....	30
Padding.....	30
Ancho y Alto.....	34
Color de texto.....	36
Color de fondo.....	38
Radio de borde.....	40

Flex Wrap.....	41
Grosor de fuente.....	42
Margin.....	43
Posicion.....	46
Align Items.....	47
Color de borde.....	48
Sombra de caja.....	50
Gap.....	51
Overflow.....	53
Alineacion de texto.....	54
Justify Content.....	55
Opacidad.....	56
Espacio en blanco.....	58
Z-Index.....	59
Transiciones.....	60
Mixins de layout.....	62
Grid.....	62
Flexbox.....	66
Breakpoints.....	67
Referencia.....	70
Funciones.....	70
Referencia API.....	72
Arquitectura.....	90
Ejemplos.....	97

Empieza aquí

KatanaKIT CSS

Por que KatanaKIT CSS

KatanaKIT CSS es un framework de **SCSS puro** inspirado en el enfoque utility-first de Tailwind CSS, pero sin una canalizacion de compilacion que no controles. Todo se genera en tiempo de compilacion a partir de mapas de tokens de disenio con `!default`:

- **Tokens de disenio** — fuentes, espaciado, radios, sombras, capas z, duraciones y funciones de temporizacion como mapas SCSS, emitidos como propiedades personalizadas de CSS.
- **Sistema de color** — 6 paletas x 7 tonos + colores especiales, con clases de utilidad, variantes hover y un tema oscuro automatico.
- **Mixins de layout** — breakpoints (7 niveles, mobile-first), un conjunto completo de CSS Grid y helpers de flexbox.
- **Clases de utilidad** — espaciado con padding y margin direccionales (`.p-*`, `.px-*`, `.py-*`, `.pt-*` ... `.m-*`, `.mx-*`, `.my-*` ...), `gap-*/gap-x-*/gap-y-*`, tamanos de texto (`.text-xs` ... `.text-4xl`) y anchos de borde (`.border`, `.border-0/2/4/8`), mas dimensiones, flex, efectos, tipografia y layout generados desde mapas `!default`.
- **Preflight extendido** — un reset estilo Tailwind que normaliza tipografia, listas, enlaces, formularios y media sobre valores por defecto basados en tokens.
- **Composicion al estilo @apply** — un pequeno sistema de registro para componer utilidades dentro de tus componentes.

Instalacion rapida

```
npm install katanakit-css
```

```
// Hoja de estilos completa (reset + tokens + utilidades)
@use "katanakit-css/src/scss/main";
```

```
// O compone tu propia hoja a partir de modulos
@use "katanakit-css/src/scss/functions" as f;
```

```
@use "katanakit-css/src/scss/variables" as v;  
@use "katanakit-css/src/scss/mixins" as m;
```

Consulta Primeros pasos para la guía completa y la Referencia API para cada función, mixin y mapa.

Playground

El repositorio incluye una demo con Vite (`yarn dev -> http://localhost:4321`) con un botón de **selector de version**: alterna entre el SCSS en vivo (HMR) y las compilaciones del framework publicadas, compiladas por `yarn build:versions` en `public/versions/`.

Primeros pasos

Esta guía te acompaña en la instalación de `katanakit-css` y la escritura de tus primeros estilos. Asume familiaridad básica con la sintaxis de módulos de Sass (`@use`). Si eres nuevo en el framework, revisa la Introducción primero; esta guía profundiza más en las opciones de configuración concretas.

La superficie API completa y verificada por código vive en la Referencia API.

Requisitos previos

- **Node.js** (≥ 20 es un límite seguro) para instalar el paquete.
- Un **compilador Sass** (Dart Sass es con lo que se prueba este proyecto) cuando consumas el código fuente SCSS. `sass` es un `devDependency` del repositorio, pero es tu responsabilidad en los proyectos dependientes.
- La **hoja de estilos precompilada** (`dist/css/katanakit.css`) no necesita herramientas adicionales.

No hay tiempo de ejecución: `katanakit-css` es SCSS/CSS puro de tiempo de compilación.

Instalación

```
npm install katanakit-css  
# o  
yarn add katanakit-css
```

Elegir un punto de entrada

Entrada	Que obtienes
dist/css/katanakit.css	Hoja completa compilada y minificada (reset + tokens + utilidades).
katanakit-css/src/scss/main	Hoja completa como SCSS que tu compilas (permite prefijos de navegador, purga personalizada, tree-shaking de modulos SCSS).
katanakit-css/src/scss/*	Modulos individuales que compones en una hoja personalizada.

Todas las importaciones SCSS usan @use en **minúsculas** con un namespace explicito. No hay @import heredado en ningun lugar.

Ruta 1 — CSS precompilado

Enlazalo o importalo:

```
<link rel="stylesheet" href="/node_modules/dist/css/katanakit.css" />
```

O importalo desde tu entrada de JavaScript en una aplicacion basada en bundler:

```
import "katanakit-css/dist/css/katanakit.css";
```

La hoja ya contiene todas las clases de utilidad que el framework puede generar, asi que puedes empezar a escribir markup inmediatamente:

```
<body class="bg-neutral-100 text-neutral-500">
  <main class="p-8">
    <h1 class="text-2xl mb-4">Hola KatanaKIT</h1>
    <div class="card"><!-- estilado por tu propio CSS de componente -->
  </div>
</main>
</body>
```

Compensacion: la hoja completa es comoda, pero incluye todas las utilidades. Para un resultado mas pequeno, usa la Ruta 2 o 3 y deja que tu compilacion (o PurgeCSS, como hace la demo) elimine las clases no utilizadas.

Ruta 2 — compilar la entrada SCSS completa

```
// styles/main.scss
```

```
@use "katanakit-css/src/scss/main";
```

Añade `styles/main.scss` a tu canalización de Sass. Esto compila el mismo contenido que la hoja precompilada: un reset preflight al estilo Tailwind, propiedades personalizadas de tokens en `:root`, propiedades personalizadas de color y utilidades, y todos los generadores de utilidades.

Para usar el registro `@apply` o los mixins en tu propio SCSS, carga los parciales también:

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.card {  
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-white);  
  
  @include m.md {  
    @include m.apply(flex-row, gap-6);  
  }  
}
```

Ruta 3 — componer una hoja personalizada

Solo pagas por los módulos que cargas. Cada parcial es importable; todos los mapas de tokens son `!default`.

```
// styles/theme.scss
```

```
@use "katanakit-css/src/scss/reset";
```

```
@use "katanakit-css/src/scss/variables" as v;
```

```
@use "katanakit-css/src/scss/functions" as f;
```

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
@use "katanakit-css/src/scss/utilities" as u;
```

```
// Tokens y variables de color
```

```
@include v.generate-css-tokens();
```

```
@include v.generate-css-vars();
```

```
// Utilidades de color: texto, fondos y hover text/bg
```

```
@include v.all-utilities();
```

```
// Generadores de utilidades (opt-in)
@include u.generate-all-utilities();
```

Recuerda: con solo cargar el modulo utilities se emiten automaticamente las clases de espaciado — padding y margin en todas las direcciones (.p-*, .px-*, .py-*, .pt-*, .m-*, .mx-*, .my-*, ...) mas gap-*/gap-x-*/gap-y-* — tamanos de texto (.text-xstext-4xl) y anchos de borde (.border, .border-0/2/4/8), asi como .gap, .shadow y .rounded. generate-all-utilities() anade clases de dimensiones, flex, efectos y layout. Omite los generadores que no necesites.

Sobreescribir tokens de disen0

Los mapas de tokens son !default, asi que los sobreescribes con una clausula with de Sass. Como un modulo solo puede configurarse una vez por compilacion, configura variables **antes** de que cualquier otro lo cargue:

```
// styles/tokens.scss – configurar primero
@use "katanakit-css/src/scss/variables" with (
  $font-families: (
    "sans-serif": (system-ui, -apple-system, "Segoe UI", sans-serif),
    "serif": (Georgia, "Times New Roman", serif),
    "mono": (ui-monospace, "SFMono-Regular", Menlo, monospace),
  ),
  $spacing-scale: (
    "0": 0, "1": 0.25rem, "2": 0.5rem, "3": 0.75rem, "4": 1rem,
    "6": 1.5rem, "8": 2rem, "12": 3rem, "16": 4rem, "20": 5rem,
  ),
  $breakpoint-sizes: ("xs": 0, "sm": 600px, "md": 900px, "lg": 1200px,
    "xl": 1440px, "2xl": 1600px, "3xl": 1920px),
);
```

```
// styles/main.scss – luego consumir
@use "tokens";
@use "katanakit-css/src/scss/main";
```

Los mapas de utilidades (_utilities.scss) se configuran de la misma manera si los cargas directamente, por ejemplo \$sizing-map, \$spacing-map, \$radius-map, \$shadow-map, \$z-layers-map, \$opacity-values, \$font-weight-values, etc.

Colores y tema oscuro

```
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();
@include v.theme("dark"); // paletas invertidas bajo :root[data-theme="dark"]

<html data-theme="dark">
  <body class="bg-neutral-100 text-neutral-500">
    <!-- neutral 100 es ahora el tono mas oscuro cuando el tema oscuro
    esta activo -->
  </body>
</html>
```

Las funciones te dan los colores crudos en tiempo de compilacion: `v.get-color("info", 300)`, `v.get("success")`, `v.alpha("danger", 500, 0.4)`.

Siguientes pasos

- Referencia API — lista completa de funciones, mixins y mapas por modulo.
- Arquitectura — grafo de modulos y flujo de compilacion.
- Hoja de ruta — trabajo enviado y planificado.
- [Demo](#) — ejecuta `yarn dev` e inspecciona `index.html` + `demo/main.js` para un ejemplo funcional de todo lo anterior.

Conceptos básicos

Tokens de disenio

Cada decision de disenio en KatanaKIT CSS vive en un conjunto de mapas SCSS con `!default` en `src/scss/_variables.scss`. Como los mapas son `!default`, los sobreescribes **una vez por compilacion** con una clausula `with` de Sass, y tanto los accesores de tokens como el CSS generado se actualizan en consecuencia.

Todos los mapas de tokens se consumen a traves del modulo `variables`:

```
@use "katanakit-css/src/scss/variables" as v;
```


Mapas de tokens

Mapa	Claves	Alimenta
\$font-families	sans-serif, serif, mono	<code>v.font-family()</code>
\$shadow-sizes	default, sm, md, lg, xl, 2xl, inner	<code>v.shadow()</code>
\$container-sizes	sm, md, lg, xl, 2xl, 3xl, 4xl, 5xl, 6xl	<code>v.container()</code>
\$breakpoint-sizes	xs, sm, md, lg, xl, 2xl, 3xl	<code>v.breakpoint()</code>
\$spacing-scale	0, 1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24, 32	<code>v.spacing()</code>
\$spacing-map	escala completa incl. fracciones (05, 1-5, 2-5, 3-5, base, 9, 11, 14, ... 96)	clases <code>.p-*</code> <code>.m-*</code> <code>.gap-*</code>
\$radius	default, sm, md, lg, xl, 2xl, 3xl, full	<code>v.radius()</code>
\$z-layers	auto, 0, 10, 20, 30, 40, 50, dropdown, sticky, fixed, modal, popover, tooltip	<code>v.z()</code>
\$transition-durations	75, 100, 150, 200, 300, 500, 700, 1000	<code>v.duration()</code>
\$transition-easings	linear, in, out, in-out	<code>v.ease()</code>

Dos mapas de espaciado coexisten a proposito:

- \$spacing-scale es la **escala semantica** usada por el accesador `v.spacing()` y expuesta como propiedades personalizadas `--spacing-*`.
- \$spacing-map es la **escala de clases** — la fuente unica de verdad para las clases de utilidad `.p-*`, `.m-*` y `.gap-*`. Anade pasos fraccionarios (05, 1-5, 2-5, 3-5), base (1px) y los pasos intermedios (7, 9, 11, 14, 28, ... 96) que los generadores de clases emiten.

Funciones de acceso

Cada accesador valida su clave en tiempo de compilacion y lanza un @error que enumera las claves validas cuando te equivocas.

Firma	Devuelve	Ejemplo
<code>v.font-family(\$name)</code>	lista de fuentes	<code>v.font-family("sans-serif")</code>
<code>v.shadow(\$size: "default")</code>	lista de sombras	<code>v.shadow("md")</code>
<code>v.container(\$size)</code>	longitud	<code>v.container("lg") -> 32rem</code>
<code>v.breakpoint(\$name)</code>	longitud en px	<code>v.breakpoint("md") -> 768px</code>
<code>v.spacing(\$size)</code>	longitud (numero o clave string)	<code>v.spacing(4) -> 1rem</code>
<code>v.radius(\$size: "default")</code>	longitud	<code>v.radius("xl") -> 0.75rem</code>
<code>v.z(\$layer)</code>	numero o auto	<code>v.z("modal") -> 1040</code>
<code>v.duration(\$ms)</code>	duracion	<code>v.duration(200) -> 200ms</code>
<code>v.ease(\$name: "in-out")</code>	funcion de temporizacion	<code>v.ease("out")</code>

```
@use "katanakit-css/src/scss/variables" as v;
```

```
.card {
  padding: v.spacing(4);
  border-radius: v.radius("default");
  box-shadow: v.shadow("lg");
  font-family: v.font-family("sans-serif");
}

.overlay {
  z-index: v.z("modal");
  transition: opacity v.duration(200) v.ease("out");
}
```

Generar propiedades personalizadas CSS

`generate-css-tokens()` emite las familias de tokens como propiedades personalizadas en `:root`. Pasa `false` para omitir una familia:

```
@mixin generate-css-tokens(
  $fonts: true, $shadows: true, $containers: true, $spacing: true,
```

```

    $radius-tokens: true, $z: true, $durations: true, $easings: true
  )

@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-tokens();

// Solo fuentes y sombras:
@include v.generate-css-tokens($containers: false, $spacing: false);

```

Resultado compilado:

```

:root {
  --font-sans-serif: ui-sans-serif, system-ui, -apple-system, ...;
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0
0 / 0.1);
  --container-xl: 36rem;
  --spacing-4: 1rem;
  --radius-full: 9999px;
  --z-tooltip: 1060;
  --duration-200: 200ms;
  --ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
}

```

Los nombres de las variables no llevan prefijo de marca: --font-*, --shadow-*, --container-*, --spacing-*, --radius-*, --z-*, --duration-* y --ease-*.

Sobreescribir tokens

Configura el modulo con with **antes** de que cualquier otro lo cargue; un modulo solo puede configurarse una vez por compilacion:

```

// styles/tokens.scss – configurar primero
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-scale: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "3": 1.5rem,
    "4": 2rem
  ),
  $radius: (
    "default": 0.5rem,
    "full": 9999px
  )
);

```

```

    ),
    $breakpoint-sizes: (
      "xs": 0,
      "sm": 600px,
      "md": 900px,
      "lg": 1200px,
      "xl": 1440px
    )
  );

// styles/main.scss – luego consumir
@use "tokens";
@use "katanakit-css/src/scss/main";

```

Los accesadores, las propiedades personalizadas --spacing-* y las clases de utilidad que derivan de los mapas configurados toman todos los nuevos valores.

Paginas relacionadas

- Colores — las paletas de color y sus accesadores.
- Breakpoints — niveles de breakpoint responsivos.
- Funciones — funciones puras auxiliares.

Colores

KatanaKIT CSS incluye **6 paletas con 7 tonos cada una** (100 mas claro -> 700 mas oscuro) mas cuatro colores especiales. Todo vive en el modulo variables y se emite como clases de utilidad, propiedades personalizadas CSS y variantes hover.

```
@use "katanakit-css/src/scss/variables" as v;
```

Paletas

Cada paleta sigue la misma rampa de luminosidad: 100-300 son tonos de fondo, 400 es el tono de marca vivid, 500-700 son tonos de texto.

Neutral

Purple

Info

Warning

Danger

Success

Colores especiales

current tambien es un color especial (currentColor), pero no tiene una muestra visible propia; se resuelve al valor color del elemento.

Valores de tono

Almacenados como hsl(tono, saturacion%, luminosidad%):

Paleta	100	200	300	400	500	600	700
neutral	0 0% 93%	0 0% 86%	0 0% 71%	0 0% 50%	0 0% 35%	0 0% 24%	0 0% 12%
purple	269 60% 93%	269 70% 86%	269 70% 71%	269 66% 50%	269 66% 35%	269 60% 24%	269 50% 12%
info	215 95% 93%	215 95% 86%	215 95% 71%	215 95% 50%	215 95% 35%	215 95% 24%	215 95% 12%
warning	49 100% 93%	49 100% 86%	49 100% 71%	49 100% 50%	49 100% 35%	49 90% 24%	49 70% 12%
danger	3 95% 93%	3 95% 86%	3 95% 71%	3 95% 50%	3 95% 35%	3 95% 24%	3 95% 12%
success	147 95% 93%	147 95% 86%	147 95% 71%	147 95% 50%	147 95% 35%	147 95% 24%	147 95% 12%

Colores especiales: black hsl(0, 0%, 3%). white hsl(0, 0%, 98%). transparent transparent. current currentColor.

Funciones de acceso

Firma	Devuelve
v.get-color(\$name, \$shade: 300, \$alpha: 1)	Un tono de paleta (o un color especial cuando \$name es especial). Aplica alfa cuando \$alpha != 1.
v.get(\$name, \$alpha: 1)	Abreviatura: get-color(\$name, 500,

Firma	Devuelve
	<code>\$alpha</code>).
<code>v.alpha(\$name, \$shade: 500, \$alpha: 0.5)</code>	Abreviatura para variantes transparentes.
<pre>@use "katanakit-css/src/scss/variables" as v; color: v.get-color("neutral", 500); // hsl(0, 0%, 35%) color: v.get("info"); // tono 500: hsl(215, 95%, 35%) border: 1px solid v.alpha("warning", 400, 0.25); background: v.alpha("purple", 100, 0.4);</pre>	

Nombres de paleta o tono invalidos lanzan un @error en tiempo de compilacion.

Generar variables CSS

`generate-css-vars()` emite una propiedad personalizada por tono mas los especiales:

```
@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();

:root {
  --neutral-100: hsl(0, 0%, 93%);
  /* ... cada paleta y tono ... */
  --info-500: hsl(215, 95%, 35%);
  --black: hsl(0, 0%, 3%);
  --white: hsl(0, 0%, 98%);
  --transparent: transparent;
  --current: currentColor;
}

// Solo la paleta info, sin especiales:
@include v.generate-css-vars($palettes: "info", $include-special:
false);

// Un selector raiz personalizado:
@include v.generate-css-vars($root: ".theme-brand");
```

`$palettes` acepta null (todas), un nombre, una lista de nombres, o un mapa emitido tal cual; este es el mecanismo que usa el tema oscuro para publicar paletas invertidas.

Clases de utilidad

Los mixins de color generan las clases que usas en el markup:

Mixin	Clases
<code>v.text-utilities()</code>	<code>.text-{palette}-{100...700}</code> + <code>.text-black .text-white .text-transparent .text-current</code>
<code>v.bg-utilities()</code>	<code>.bg-{palette}-{100...700}</code> + <code>.bg-black .bg-white .bg-transparent .bg-current</code>
<code>v.border-utilities()</code>	<code>.border-{palette}-{100...700}</code> + los cuatro especiales
<code>v.hover-utilities()</code>	<code>.hover-text-*:hover</code> y <code>.hover-bg-*:hover</code>
<code>v.all-utilities()</code>	Los cuatro mixins a la vez (llamado por <code>main.scss</code>).

Consulta Color de texto, Color de fondo y Color de borde para las tablas completas de clases.

Breakpoints

Los breakpoints impulsan los mixins responsivos. Los niveles viven en dos lugares que deben mantenerse sincronizados: `v.$breakpoint-sizes` (accesador de tokens) y `m.$breakpoints` (motor de mixins). Ambos son mapas !default con los mismos valores por defecto.

Nivel	Ancho
xs	0
sm	640px
md	768px
lg	1024px
xl	1280px
2xl	1536px
3xl	1920px

```
@use "katanakit-css/src/scss/mixins" as m;  
@use "katanakit-css/src/scss/variables" as v;
```

// El accesador de tokens devuelve valores crudos para tus propias media queries:

```
$tablet: v.breakpoint("md"); // 768px
```

Las claves que contienen digitos (2x1, 3x1) son nombres de primera clase; pasalas siempre como strings.

El mixin generico

`breakpoint($from, $direction: up, $to: null)` abre una consulta `@media`. `$from` (y `$to`) aceptan un nombre de nivel o un numero px en bruto. `bp()` es un alias que reenvia sus argumentos.

Direccion	Consulta media	Requiere \$to
up	(min-width: X)	no
down	(max-width: calc(X - 0.02px))	no
only	(min-width: X) and (max-width: calc(Y - 0.02px))	si
between	(min-width: X) and (max-width: Y)	si

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
@include m.breakpoint("md", up) {  
  .card { display: grid; }  
}
```

```
@include m.bp("lg", down) { /* alias, igual que breakpoint("lg", down) */ }
```

```
@include m.breakpoint("sm", only, "md") { /* solo tablets */ }
```

```
@include m.breakpoint(900px, between, 1200px) { /* los valores en bruto tambien funcionan */ }
```

Mixins de direccion (mobile-first)

Mixins de atajo para la direccion up, incluyendo alias `xx1` / `xxx1` para las claves con digitos:


```

@include m.xs    { /* >= 0      */ }
@include m.sm    { /* >= 640px */ }
@include m.md    { /* >= 768px */ }
@include m.lg    { /* >= 1024px */ }
@include m.xl    { /* >= 1280px */ }
@include m.xxl   { /* >= 1536px (2x1) */ }
@include m.xxxl  { /* >= 1920px (3x1) */ }

```

Mixins down (desktop-first)

```

@include m.xs-down { /* <= 0 */ }
@include m.sm-down { /* <= 639.98px */ }
@include m.md-down { /* <= 767.98px */ }
@include m.lg-down { /* <= 1023.98px */ }
@include m.xl-down { /* <= 1279.98px */ }
@include m.x2l-down { /* <= 1535.98px (2x1) */ }
@include m.x3l-down { /* <= 1919.98px (3x1) */ }

```

// Alias canonicos para las claves con digitos:

```

@include m.xxl-down { /* = x2l-down */ }
@include m.xxxl-down { /* = x3l-down */ }

```

Combinando ambas direcciones

Una rejilla de tarjetas responsiva tipica, mobile-first y luego limitada en el extremo superior:

```

@use "katanakit-css/src/scss/mixins" as m;

.grid {
  display: grid;
  grid-template-columns: 1fr;

  @include m.sm { grid-template-columns: repeat(2, 1fr); }
  @include m.lg { grid-template-columns: repeat(4, 1fr); }
}

```

Consulta la referencia del mixin Breakpoints para la referencia tecnica completa del mixin generico.

Consultas de características

Mas alla de los anchos, el modulo incluye atajos para media features:

Mixin	Consulta media
m.portrait	(orientation: portrait)
m.landscape	(orientation: landscape)
m.reduced-motion	(prefers-reduced-motion: reduce)
m.hoverable	(hover: hover) and (pointer: fine)
m.touch	(hover: none) and (pointer: coarse)
m.dark-mode	(prefers-color-scheme: dark)
m.light-mode	(prefers-color-scheme: light)

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.hero-animation {
  animation: slide-up 300ms ease;

  @include m.reduced-motion {
    animation: none;
  }

  @include m.touch {
    // Blancos de toque mas grandes en dispositivos tactiles
    padding: 1.5rem;
  }
}
```

m.dark-mode y m.light-mode siguen la preferencia del **sistema**. Para el tema basado en clases, consulta Modo oscuro.

Sobreescribir los niveles

Como ambos mapas son !default, configuralos antes del primer uso:

```
@use "katanakit-css/src/scss/variables" as v with (
  $breakpoint-sizes: (
    "xs": 0,
    "sm": 600px,
    "md": 900px,
    "lg": 1200px,
    "xl": 1440px,
```

```

        "2xl": 1600px
    )
);
@use "katanakit-css/src/scss/mixins" as m with (
    $breakpoints: (
        "xs": 0,
        "sm": 600px,
        "md": 900px,
        "lg": 1200px,
        "xl": 1440px,
        "2xl": 1600px
    )
);

```

Modo oscuro

KatanaKIT CSS genera un tema oscuro basado en clases de forma gratuita:

`v.theme("dark")` toma las seis paletas, **invierte cada tono** (100 <-> 700, 200 <-> 600, 300 <-> 500) y publica el resultado como propiedades personalizadas bajo `:root[data-theme="dark"]`.

La inversion se aplica a las **propiedades personalizadas** (`--neutral-100, ...`). Las clases de utilidad de color (`.bg-neutral-100, .text-neutral-500, ...`) compilan a valores literales y son independientes del tema; usa las variables en tus propias reglas cuando necesites estilos conscientes del tema.

Activar el tema oscuro

```

@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars(); // valores por defecto claros en :root
@include v.theme("dark");      // paletas invertidas en :root[data-theme="dark"]

:root[data-theme="dark"] {
    --neutral-100: hsl(0, 0%, 12%); /* era 700 */
    --neutral-200: hsl(0, 0%, 24%); /* era 600 */
    /* ... */
    --neutral-700: hsl(0, 0%, 93%); /* era 100 */
}

```

Luego cambia el tema estableciendo el atributo:

```

<html data-theme="dark">
  <body class="surface text-body">
    <!-- .surface/.text-body son TUS reglas que consumen
         var(--neutral-100) / var(--neutral-500): bajo el
         tema oscuro esas variables ahora contienen los tonos
invertidos -->
    </body>
  </html>

// Alternar entre claro y oscuro
document.documentElement.setAttribute("data-theme", "dark");
document.documentElement.removeAttribute("data-theme");

```

Como funciona la inversion

La inversion es una transformacion pura de mapa: \$inverted: 800 - \$shade.

Tono claro	Tono oscuro
100	700
200	600
300	500
400	400
500	300
600	200
700	100

Por esto el vocabulario tonal se mantiene estable: bg-neutral-100 es siempre “la superficie”, text-neutral-500 es siempre “el texto legible”; los valores HSL reales se intercambian por debajo.

Construir sobre las variables

Las variables del tema oscuro son propiedades personalizadas normales, así que compónlas libremente:

```

@use "katanakit-css/src/scss/variables" as v;

@include v.generate-css-vars();
@include v.theme("dark");

body {
  background-color: var(--neutral-100);

```

```

    color: var(--neutral-500);
}

.brand {
    color: var(--purple-400);
}

```

Observa que las **clases de utilidad** de color (`v.text-utilities()` etc.) compilan a valores literales, no a referencias `var()`. Si necesitas clases conscientes del tema, usa las propiedades personalizadas en tus propias reglas, como se muestra arriba.

Sobreescribir la paleta oscura

`theme()` acepta un mapa `$overrides` que se fusiona sobre el resultado invertido:

```

@include v.theme(
  "dark",
  (
    "neutral": (
      100: hsl(0, 0%, 8%),
      500: hsl(0, 0%, 88%)
    )
  )
);

```

Combinar con la preferencia del sistema

Combina el tema basado en atributo con la consulta de característica `m.dark-mode` para sincronizar `color-scheme` (barras de desplazamiento, controles de formulario) con la preferencia del sistema:

```

@use "katanakit-css/src/scss/variables" as v;
@use "katanakit-css/src/scss/mixins" as m;

@include v.generate-css-vars();
@include v.theme("dark");

@include m.dark-mode {
  :root {
    color-scheme: dark;
  }
}

```

Para seguir la preferencia del sistema automaticamente, establece data-theme desde JavaScript:

```
const mq = window.matchMedia("(prefers-color-scheme: dark)");
const apply = () =>
  document.documentElement.toggleAttribute("data-theme", mq.matches);

apply();
mq.addEventListener("change", apply);
```

Paginas relacionadas

- Colores — paletas y generate-css-vars().
- Breakpoints — la consulta de caracteristica m.dark-mode.
- Color de texto y Color de fondo — las clases de utilidad de color.

@apply

KatanaKIT CSS incluye un pequeno sistema al estilo @apply para que puedas componer utilidades dentro de tus componentes **sin** reimplementarlas:

```
@use "katanakit-css/src/scss/mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-white);
}
```

Dos mixins alimentan el sistema:

Mixin	Comportamiento
m.register-utility(\$name, \$styles)	Registra una utilidad con nombre. \$styles es un mapa de declaraciones CSS.
m.apply(\$utilities...)	Emite cada declaracion registrada para cada nombre, en orden. Lanza un @error nombrando la utilidad problematica cuando una no esta registrada.

Uso basico

```
@use "katanakit-css/src/scss/mixins" as m;
```

```

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
white);

  &:hover {
    @include m.apply(shadow-lg);
  }
}

.button {
  @include m.apply(
    inline-flex,
    items-center,
    justify-center,
    px-4,
    py-2,
    rounded-md,
    font-semibold,
    transition-colors
  );

  &:hover {
    @include m.apply(bg-white, text-black);
  }
}

```

Compila a declaraciones puras; sin tiempo de ejecucion, sin clases duplicadas en el resultado:

```

.card {
  display: flex;
  flex-direction: column;
  padding: 1rem;
  border-radius: 0.5rem;
  box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0
0 / 0.1);
  background-color: hsl(0, 0%, 98%);
}

```

Registrar tus propias utilidades

El registro esta abierto; registra utilidades especificas del proyecto antes de usarlas:

```

@use "katanakit-css/src/scss/mixins" as m;

@include m.register-utility("fade-in", (animation: fade-in 300ms
ease));

.hero {
  @include m.apply(fade-in, p-8, text-2xl);
}

```

Utilidades registradas automaticamente

Cargar el modulo mixins registra un gran conjunto de nombres; la mayoria derivados de los **mismos mapas** que impulsan los generadores de clases, por lo que los nombres coinciden exactamente con los nombres de las clases (p-4, m-2, gap-4, rounded-lg, shadow-md, z-10, duration-200, ease-out, opacity-50, ...).

Categoria	Nombres registrados
Display	block, inline-block, inline, flex, inline-flex, grid, hidden, table, table-row, table-cell, contents, inline-grid
Flexbox	flex-row, flex-row-reverse, flex-col, flex-col-reverse, flex-wrap, flex-nowrap, items-start, items-center, items-end, items-stretch, justify-start, justify-center, justify-end, justify-between, justify-around, justify-evenly
Espaciado	para cada clave de \$spacing-map: p-* px-* py-* pt-* pr-* pb-* pl-*, m-* mx-* my-* mt-* mr-* mb-* ml-*, gap-* gap-x-* gap-y-*
Tipografia	text-xs ... text-4xl, text-left, text-center, text-right, font-thin ... font-black
Dimensiones	w-full, w-screen, w-auto, h-full, h-screen, h-auto
Bordes	border, border-0, border-2, border-4, border-8, rounded-none ... rounded-

Categoría	Nombres registrados
Efectos	full, rounded shadow-none ... shadow-2xl + shadow + shadow-inner, opacity-0 ... opacity-100, z-auto ... z-tooltip, duration-75 ... duration-1000, ease-linear ... ease-in-out
Posicion	static, fixed, absolute, relative, sticky
Overflow	overflow-auto, overflow-hidden, overflow-scroll, overflow-visible
Colores	text-white, text-black, bg-white, bg-black, bg-transparent
Espacio en blanco	whitespace-nowrap, whitespace-pre, whitespace-normal, wrap-break-word

Utilidades solo en el registro

Importante. Registrar una utilidad para apply **no** crea una clase CSS. Un pequeño conjunto de nombres existe solo dentro del registro; puedes usarlos con `apply()`, pero nunca aparecerán como clases en el CSS compilado:

- flex-1, flex-auto, flex-none
- grid-cols-1, grid-cols-2, grid-cols-3, grid-cols-4, grid-cols-6, grid-cols-12
- col-span-full
- transition, transition-colors, transition-opacity, transition-transform

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.hero-grid {
  @include m.apply(grid, grid-cols-12, gap-4);

  .main { @include m.apply(col-span-full); }
  .aside { @include m.apply(flex-1); }
}
```

```
<!-- Incorrecto: .flex-1 NO se emite como clase por el framework -->
<div class="flex-1">...</div>
```

Errores

Aplicar un nombre no registrado falla de forma visible en tiempo de compilacion:

```
@include m.apply(flex, flex-col, text-5xl);  
// Error: apply(): utilidad 'text-5xl' no registrada.  
//      Usa register-utility() primero.
```

Paginas relacionadas

- Primeros pasos — componer hojas con modulos.
- Ejemplos — el ejemplo de boton/tarjeta completo.

Utilidades

Ancho de borde

Utilidades para controlar el ancho de los bordes de un elemento. `.border` (1px solido) se emite automaticamente con el modulo `utilities`; los demas anchos provienen de `$border-width-map`.

Referencia rapida

Clase	Propiedades
<code>.border</code>	<code>border-width: 1px; border-style: solid;</code>
<code>.border-0</code>	<code>border-width: 0;</code>
<code>.border-2</code>	<code>border-width: 2px;</code>
<code>.border-4</code>	<code>border-width: 4px;</code>
<code>.border-8</code>	<code>border-width: 8px;</code>

Uso basico

```
<div class="border border-purple-500 ...">border</div>  
<div class="border-2 border-purple-500 ...">border-2</div>  
<div class="border-4 border-purple-500 ...">border-4</div>
```

Los anchos son independientes de los colores; anade un color de Color de borde o se aplica el valor por defecto (`currentColor`).

Personalizar

```
@use "katanakit-css/src/scss/utilities" as u with (  
  $border-width-map: (  
    0: 0px,  
    1: 1px,  
    2: 2px,  
    4: 4px,  
    8: 8px,  
  )  
)
```

```
    "0": 0,  
    "2": 2px,  
    "4": 4px  
  )  
);
```

Display

Utilidades para controlar el tipo de caja de display de un elemento. Generadas desde el mapa `$display-values` (un mapa para que `hidden` pueda mapear a `none`).

Referencia rapida

Clase	Propiedades
<code>.block</code>	<code>display: block;</code>
<code>.inline-block</code>	<code>display: inline-block;</code>
<code>.inline</code>	<code>display: inline;</code>
<code>.flex</code>	<code>display: flex;</code>
<code>.inline-flex</code>	<code>display: inline-flex;</code>
<code>.grid</code>	<code>display: grid;</code>
<code>.inline-grid</code>	<code>display: inline-grid;</code>
<code>.table</code>	<code>display: table;</code>
<code>.table-row</code>	<code>display: table-row;</code>
<code>.table-cell</code>	<code>display: table-cell;</code>
<code>.hidden</code>	<code>display: none;</code>
<code>.contents</code>	<code>display: contents;</code>

Uso basico

Establece el tipo de display de un elemento:

```
<div class="block ...">block</div>  
<div class="inline-block ...">inline-block</div>  
<div class="inline ...">inline</div>
```

Usar flex y grid

`flex` y `grid` crean los contenedores sobre los que funciona cada utilidad de `flexbox` y `grid`:

```
<div class="flex gap-2">...</div>
<div class="grid gap-2" style="grid-template-columns: repeat(3,
1fr)">...</div>
```

Ocultar elementos

Usa hidden para eliminar un elemento del flujo del documento por completo:

```
<div class="flex gap-2">
  <div class="p-4 ...">visible</div>
  <div class="hidden p-4 ...">hidden</div>
</div>
```

A diferencia de `opacity-0`, hidden no ocupa espacio ni recibe eventos de puntero.

Usar displays de tabla

Las utilidades `table-*` te permiten aplicar semántica de layout de tabla a cualquier estructura:

```
<div class="table">
  <div class="table-row">
    <div class="table-cell p-2">Celda A</div>
    <div class="table-cell p-2">Celda B</div>
  </div>
</div>
```

Personalizar

Sobreescribe el mapa `$display-values` en el módulo mixins antes de generar las utilidades de layout:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $display-values: (
    block: block,
    flex: flex,
    grid: grid,
    hidden: none
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Direccion flex

Utilidades para controlar la direccion de los elementos flex. Requiere un contenedor flex (`.flex` o `.inline-flex`; consulta Display).

Referencia rapida

Clase	Propiedades
<code>.flex-row</code>	<code>flex-direction: row;</code>
<code>.flex-row-reverse</code>	<code>flex-direction: row-reverse;</code>
<code>.flex-col</code>	<code>flex-direction: column;</code>
<code>.flex-col-reverse</code>	<code>flex-direction: column-reverse;</code>

Uso basico

Organiza los elementos en una fila (por defecto) o una columna:

```
<div class="flex flex-row gap-2">...</div>
<div class="flex flex-col gap-2">...</div>
```

Invertir la direccion

`flex-row-reverse` y `flex-col-reverse` mantienen el mismo eje de layout pero invierten el orden visual de los elementos:

```
<div class="flex flex-row-reverse gap-2">...</div>
```

Personalizar

Las clases de direccion se generan desde `$flex-direction-map` en el modulo mixins:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $flex-direction-map: (
    "row": row,
    "col": column
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Tamano de fuente

Utilidades para controlar el tamaño de fuente de un elemento, generadas desde `$font-size-map`. Estas clases se emiten automáticamente cuando se carga el módulo `utilities`; no se necesita llamar a un generador.

Referencia rápida

Clase	Propiedades
<code>.text-xs</code>	<code>font-size: 0.75rem;</code>
<code>.text-sm</code>	<code>font-size: 0.875rem;</code>
<code>.text-base</code>	<code>font-size: 1rem;</code>
<code>.text-lg</code>	<code>font-size: 1.125rem;</code>
<code>.text-xl</code>	<code>font-size: 1.25rem;</code>
<code>.text-2xl</code>	<code>font-size: 1.5rem;</code>
<code>.text-3xl</code>	<code>font-size: 1.875rem;</code>
<code>.text-4xl</code>	<code>font-size: 2.25rem;</code>

Uso básico

```
<p class="text-xs ...">El rapido zorro marron – text-xs</p>
<p class="text-sm ...">El rapido zorro marron – text-sm</p>
<!-- ... hasta text-4xl -->
```

Personalizar

Sobreescribe `$font-size-map` en el módulo `utilities` antes de que se cargue:

```
@use "katanakit-css/src/scss/utilities" as u with (
  $font-size-map: (
    "sm": 0.75rem,
    "base": 1rem,
    "xl": 1.5rem
  )
);
```

Padding

Utilidades para controlar el padding de un elemento.

Referencia rapida

Cada valor de \$spacing-map se emite para los siete prefijos: p-* (todos los lados), px-*/py-* (horizontal/vertical) y pt-*/pr-*/pb-*/pl-* (un lado).

Escala	p-*	px-*	py-*	pt-*	pr-*	pb-*	pl-*
0 -> 0	.p-0	.px-0	.py-0	.pt-0	.pr-0	.pb-0	.pl-0
05 -> 0.125rem	.p-05	.px-05	.py-05	.pt-05	.pr-05	.pb-05	.pl-05
base -> 1px	.p-base	.px-base	.py-base	.pt-base	.pr-base	.pb-base	.pl-base
1 -> 0.25rem	.p-1	.px-1	.py-1	.pt-1	.pr-1	.pb-1	.pl-1
1-5 -> 0.375rem	.p-1-5	.px-1-5	.py-1-5	.pt-1-5	.pr-1-5	.pb-1-5	.pl-1-5
2 -> 0.5rem	.p-2	.px-2	.py-2	.pt-2	.pr-2	.pb-2	.pl-2
2-5 -> 0.625rem	.p-2-5	.px-2-5	.py-2-5	.pt-2-5	.pr-2-5	.pb-2-5	.pl-2-5
3 -> 0.75rem	.p-3	.px-3	.py-3	.pt-3	.pr-3	.pb-3	.pl-3
3-5 -> 0.875rem	.p-3-5	.px-3-5	.py-3-5	.pt-3-5	.pr-3-5	.pb-3-5	.pl-3-5
4 -> 1rem	.p-4	.px-4	.py-4	.pt-4	.pr-4	.pb-4	.pl-4
5 -> 1.25rem	.p-5	.px-5	.py-5	.pt-5	.pr-5	.pb-5	.pl-5
6 -> 1.5rem	.p-6	.px-6	.py-6	.pt-6	.pr-6	.pb-6	.pl-6
7 -> 1.75rem	.p-7	.px-7	.py-7	.pt-7	.pr-7	.pb-7	.pl-7
8 -> 2rem	.p-8	.px-8	.py-8	.pt-8	.pr-8	.pb-8	.pl-8
9 ->	.p-9	.px-9	.py-9	.pt-9	.pr-9	.pb-9	.pl-9

Escala	p-*	px-*	py-*	pt-*	pr-*	pb-*	pl-*
2.25rem							
10 ->	.p-10	.px-10	.py-10	.pt-10	.pr-10	.pb-10	.pl-10
2.5rem							
11 ->	.p-11	.px-11	.py-11	.pt-11	.pr-11	.pb-11	.pl-11
2.75rem							
12 ->	.p-12	.px-12	.py-12	.pt-12	.pr-12	.pb-12	.pl-12
3rem							
14 ->	.p-14	.px-14	.py-14	.pt-14	.pr-14	.pb-14	.pl-14
3.5rem							
16 ->	.p-16	.px-16	.py-16	.pt-16	.pr-16	.pb-16	.pl-16
4rem							
20 ->	.p-20	.px-20	.py-20	.pt-20	.pr-20	.pb-20	.pl-20
5rem							
24 ->	.p-24	.px-24	.py-24	.pt-24	.pr-24	.pb-24	.pl-24
6rem							
28 ->	.p-28	.px-28	.py-28	.pt-28	.pr-28	.pb-28	.pl-28
7rem							
32 ->	.p-32	.px-32	.py-32	.pt-32	.pr-32	.pb-32	.pl-32
8rem							
36 ->	.p-36	.px-36	.py-36	.pt-36	.pr-36	.pb-36	.pl-36
9rem							
40 ->	.p-40	.px-40	.py-40	.pt-40	.pr-40	.pb-40	.pl-40
10rem							
44 ->	.p-44	.px-44	.py-44	.pt-44	.pr-44	.pb-44	.pl-44
11rem							
48 ->	.p-48	.px-48	.py-48	.pt-48	.pr-48	.pb-48	.pl-48
12rem							
52 ->	.p-52	.px-52	.py-52	.pt-52	.pr-52	.pb-52	.pl-52
13rem							
56 ->	.p-56	.px-56	.py-56	.pt-56	.pr-56	.pb-56	.pl-56
14rem							
60 ->	.p-60	.px-60	.py-60	.pt-60	.pr-60	.pb-60	.pl-60
15rem							
64 ->	.p-64	.px-64	.py-64	.pt-64	.pr-64	.pb-64	.pl-64
16rem							

Escala	p-*	px-*	py-*	pt-*	pr-*	pb-*	pl-*
72 -> 18rem	.p-72	.px-72	.py-72	.pt-72	.pr-72	.pb-72	.pl-72
80 -> 20rem	.p-80	.px-80	.py-80	.pt-80	.pr-80	.pb-80	.pl-80
96 -> 24rem	.p-96	.px-96	.py-96	.pt-96	.pr-96	.pb-96	.pl-96

Consulta la pagina de Tokens de disenio para la definicion completa de la escala de espaciado.

Uso basico

Añade padding a todos los lados de un elemento:

```
<div class="p-8 ..." >p-8</div>
```

Añadir padding horizontal y vertical

Controla el padding horizontal de un elemento con las utilidades px-* y el padding vertical con las utilidades py-*:

```
<div class="px-8 py-4 ..." >px-8 py-4</div>
```

Padding en un solo lado

Usa pt-*, pr-*, pb-* y pl-* para añadir padding a un solo lado:

```
<div class="pt-8 pr-4 pb-2 pl-6 ..." >pt-8 . pr-4 . pb-2 . pl-6</div>
```

Usar propiedades logicas

Cada utilidad de padding es una unica declaracion padding*, por lo que se componen libremente con el resto del framework:

```
// Las clases se generan desde $spacing-map en _utilities.scss:
@each $key, $value in $spacing-map {
  .p-#{ $key } { padding: $value; }
  .px-#{ $key } { padding-left: $value; padding-right: $value; }
  // ...
}
```

Personalizar

Sobreescribe \$spacing-map antes de importar las utilidades para cambiar toda la escala a la vez:

```
@use "katanakit-css/src/scss/variables" as v with (  
  $spacing-map: (  
    "0": 0,  
    "1": 0.5rem,  
    "2": 1rem,  
    "4": 2rem  
  )  
);
```

Ancho y Alto

Utilidades para establecer el ancho y alto de un elemento, generadas desde \$sizing-map para seis prefijos: w-*, min-w-*, max-w-*, h-*, min-h-*, max-h-*.

Referencia rapida

Clave	Valor	w-*	min-w-*	max-w-*	h-*	min-h-*	max-h-*
0	0	.w-0	.min-w-0	.max-w-0	.h-0	.min-h-0	.max-h-0
base	1px	.w-base	.min-w-base	.max-w-base	.h-base	.min-h-base	.max-h-base
05	0.125rem	.w-05	.min-w-05	.max-w-05	.h-05	.min-h-05	.max-h-05
1	0.25rem	.w-1	.min-w-1	.max-w-1	.h-1	.min-h-1	.max-h-1
2	0.5rem	.w-2	.min-w-2	.max-w-2	.h-2	.min-h-2	.max-h-2
3	0.75rem	.w-3	.min-w-3	.max-w-3	.h-3	.min-h-3	.max-h-3
4	1rem	.w-4	.min-w-4	.max-w-4	.h-4	.min-h-4	.max-h-4
5	1.25rem	.w-5	.min-w-5	.max-w-5	.h-5	.min-h-5	.max-h-5
6	1.5rem	.w-6	.min-w-6	.max-w-6	.h-6	.min-h-6	.max-h-6
8	2rem	.w-8	.min-w-8	.max-w-8	.h-8	.min-h-8	.max-h-8

Clave	Valor	w-*	min-w-*	max-w-*	h-*	min-h-*	max-h-*
			w-8	w-8		h-8	h-8
10	2.5rem	.w-10	.min-w-10	.max-w-10	.h-10	.min-h-10	.max-h-10
12	3rem	.w-12	.min-w-12	.max-w-12	.h-12	.min-h-12	.max-h-12
16	4rem	.w-16	.min-w-16	.max-w-16	.h-16	.min-h-16	.max-h-16
20	5rem	.w-20	.min-w-20	.max-w-20	.h-20	.min-h-20	.max-h-20
24	6rem	.w-24	.min-w-24	.max-w-24	.h-24	.min-h-24	.max-h-24
32	8rem	.w-32	.min-w-32	.max-w-32	.h-32	.min-h-32	.max-h-32
40	10rem	.w-40	.min-w-40	.max-w-40	.h-40	.min-h-40	.max-h-40
48	12rem	.w-48	.min-w-48	.max-w-48	.h-48	.min-h-48	.max-h-48
64	16rem	.w-64	.min-w-64	.max-w-64	.h-64	.min-h-64	.max-h-64
auto	auto	.w-auto	.min-w-auto	.max-w-auto	.h-auto	.min-h-auto	.max-h-auto
full	100%	.w-full	.min-w-full	.max-w-full	.h-full	.min-h-full	.max-h-full
screen	100vw	.w-screen	.min-w-screen	.max-w-screen	.h-screen	.min-h-screen	.max-h-screen
min	min-content	.w-min	.min-w-min	.max-w-min	.h-min	.min-h-min	.max-h-min
max	max-content	.w-max	.min-w-max	.max-w-max	.h-max	.min-h-max	.max-h-max
fit	fit-content	.w-fit	.min-w-fit	.max-w-fit	.h-fit	.min-h-fit	.max-h-fit

Nota: screen siempre se resuelve a 100vw incluso para los prefijos de altura. Si necesitas una caja de viewport a altura completa, prefiere .h-full en un padre o tu propia regla height: 100vh.

Uso basico

```
<div class="w-64 ...">w-64</div>
<div class="w-40 h-24 ...">w-40 h-24</div>
<div class="w-24 h-24 ...">w-24 h-24</div>
```

Usar porcentajes y tamanos de contenido

full es 100%, y min/max/fit mapean a los tamanos intrinsecos de CSS:

```
<div class="w-full ...">llena el ancho del padre</div>
<div class="w-fit ...">se reduce a su contenido</div>
<div class="w-max ...">crece hasta su linea mas larga</div>
```

Restringir con min y max

max-w-* y min-h-* limitan un elemento en lugar de fijarlo:

```

<article class="max-w-64">mantiene longitudes de linea
legibles</article>
```

Personalizar

Sobreescribe \$sizing-map en el modulo utilities antes de generar:

```
@use "katanakit-css/src/scss/utilities" as u with (
  $sizing-map: (
    "0": 0,
    "1": 0.25rem,
    "4": 1rem,
    "full": 100%,
    "screen": 100vw,
    "auto": auto
  )
);
```

```
@include u.get-sizing-classes();
```

Color de texto

Utilidades para controlar el color de texto de un elemento, generadas por `v.text-utilities()` a partir de las paletas de color. Las 6 paletas con 7 tonos cada una, mas los colores especiales y variantes `:hover`.

Referencia rapida

Paleta	100	200	300	400	500	600	700
neutral	.text-neutral-100	.text-neutral-200	.text-neutral-300	.text-neutral-400	.text-neutral-500	.text-neutral-600	.text-neutral-700
purple	.text-purple-100	.text-purple-200	.text-purple-300	.text-purple-400	.text-purple-500	.text-purple-600	.text-purple-700
info	.text-info-100	.text-info-200	.text-info-300	.text-info-400	.text-info-500	.text-info-600	.text-info-700
warning	.text-warning-100	.text-warning-200	.text-warning-300	.text-warning-400	.text-warning-500	.text-warning-600	.text-warning-700
danger	.text-danger-100	.text-danger-200	.text-danger-300	.text-danger-400	.text-danger-500	.text-danger-600	.text-danger-700
success	.text-success-100	.text-success-200	.text-success-300	.text-success-400	.text-success-500	.text-success-600	.text-success-700

Especial	Clase	Propiedades
black	.text-black	color: hsl(0, 0%, 3%);
white	.text-white	color: hsl(0, 0%, 98%);
transparent	.text-transparent	color: transparent;
current	.text-current	color: currentColor;

Uso basico

```
<p class="text-purple-500 ...">text-purple-500</p>  
<p class="text-white ...">text-white</p>
```

Variantes hover

Cada color de texto tambien existe como variante `:hover`, generada por `v.hover-utilities()`:

```
<a href="#" class="text-purple-600 hover-text-purple-400 ..." >Pasa el  
raton</a>
```

Variante	Paletas
<code>.hover-text-{palette}-{100...700}:hover</code>	<code>.hover-text-neutral-100hover-text-success-700</code>
<code>.hover-text-{special}:hover</code>	<code>.hover-text-black, .hover-text-white, .hover-text-transparent, .hover-text-current</code>

Personalizar

Las utilidades de color se generan desde `$colors` y `$special-colors` en el modulo variables:

```
@use "katanakit-css/src/scss/variables" as v;  
  
// Solo las paletas purple y neutral:  
@include v.text-utilities(("purple", "neutral"));  
  
// Todas las paletas, sin colores especiales:  
@include v.text-utilities($special: false);  
  
// Todo:  
@include v.all-utilities();
```

Color de fondo

Utilidades para controlar el color de fondo de un elemento, generadas por `v.bg-utilities()` a partir de las paletas de color. Las 6 paletas con 7 tonos cada una, mas los colores especiales y variantes `:hover`.

Referencia rapida

Paleta	100	200	300	400	500	600	700
neutral	<code>.bg-neutral-100</code>	<code>.bg-neutral-200</code>	<code>.bg-neutral-300</code>	<code>.bg-neutral-400</code>	<code>.bg-neutral-500</code>	<code>.bg-neutral-600</code>	<code>.bg-neutral-700</code>

Paleta	100	200	300	400	500	600	700
purple	.bg-purple-100	.bg-purple-200	.bg-purple-300	.bg-purple-400	.bg-purple-500	.bg-purple-600	.bg-purple-700
info	.bg-info-100	.bg-info-200	.bg-info-300	.bg-info-400	.bg-info-500	.bg-info-600	.bg-info-700
warning	.bg-warning-100	.bg-warning-200	.bg-warning-300	.bg-warning-400	.bg-warning-500	.bg-warning-600	.bg-warning-700
danger	.bg-danger-100	.bg-danger-200	.bg-danger-300	.bg-danger-400	.bg-danger-500	.bg-danger-600	.bg-danger-700
success	.bg-success-100	.bg-success-200	.bg-success-300	.bg-success-400	.bg-success-500	.bg-success-600	.bg-success-700

Especial	Clase	Propiedades
black	.bg-black	background-color: hsl(0, 0%, 3%);
white	.bg-white	background-color: hsl(0, 0%, 98%);
transparent	.bg-transparent	background-color: transparent;
current	.bg-current	background-color: currentColor;

Uso basico

```
<div class="bg-purple-500 text-white ...">purple-500</div>
```

El conjunto completo de muestras de cada paleta se renderiza en la pagina de Colores.

Variantes hover

Cada color de fondo tambien existe como variante :hover, generada por v.hover-utilities():

```
<button class="bg-purple-300 text-purple-700 hover-bg-purple-500 ...">Pasa el raton</button>
```

Variante	Paletas
<code>.hover-bg-{palette}-{100...700}:hover</code>	<code>.hover-bg-neutral-100hover-bg-success-700</code>
<code>.hover-bg-{special}:hover</code>	<code>.hover-bg-black, .hover-bg-white, .hover-bg-transparent, .hover-bg-current</code>

Personalizar

```
@use "katanakit-css/src/scss/variables" as v;
```

```
// Solo la paleta info:
```

```
@include v.bg-utilities("info");
```

```
// Todas las paletas, sin colores especiales:
```

```
@include v.bg-utilities($special: false);
```

```
// Todo:
```

```
@include v.all-utilities();
```

Radio de borde

Utilidades para controlar el radio de borde de un elemento, generadas desde \$radius-map. La clase base `.rounded (0.25rem)` se emite automaticamente con el modulo `utilities`.

Referencia rapida

Clase	Propiedades
<code>.rounded</code>	<code>border-radius: 0.25rem;</code>
<code>.rounded-none</code>	<code>border-radius: 0;</code>
<code>.rounded-sm</code>	<code>border-radius: 0.125rem;</code>
<code>.rounded-md</code>	<code>border-radius: 0.375rem;</code>
<code>.rounded-lg</code>	<code>border-radius: 0.5rem;</code>
<code>.rounded-xl</code>	<code>border-radius: 0.75rem;</code>
<code>.rounded-2xl</code>	<code>border-radius: 1rem;</code>
<code>.rounded-3xl</code>	<code>border-radius: 1.5rem;</code>
<code>.rounded-full</code>	<code>border-radius: 9999px;</code>

Uso basico

```
<div class="rounded-lg ...">rounded-lg</div>
<div class="rounded-full ...">rounded-full</div>
```

Hacer círculos

rounded-full con ancho y alto iguales produce un círculo:

```
<div class="w-24 h-24 rounded-full bg-purple-500"></div>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $radius-map: (
    "none": 0,
    "md": 0.375rem,
    "full": 9999px
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utililities();
```

Flex Wrap

Utilidades para controlar si los elementos flex se envuelven en multiples lineas. Requiere un contenedor flex (.flex o .inline-flex).

Referencia rapida

Clase	Propiedades
.flex-wrap	flex-wrap: wrap;
.flex-nowrap	flex-wrap: nowrap;

Uso basico

flex-wrap permite que los elementos pasen a nuevas lineas cuando el contenedor se queda sin espacio; flex-nowrap fuerza una sola linea:

```
<div class="flex flex-wrap gap-2 w-64">...</div>
```

Prevenir el envoltura

```
<div class="flex flex-nowrap">
  <span class="whitespace-nowrap">Nunca se rompe</span>
```

```
<span class="whitespace-nowrap">Se mantiene en una linea</span>
</div>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $flex-wrap-list: (wrap, nowrap)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Grosor de fuente

Utilidades para controlar el grosor de fuente de un elemento, generadas desde el mapa \$font-weight-values.

Referencia rapida

Clase	Propiedades
.font-thin	font-weight: 100;
.font-extralight	font-weight: 200;
.font-light	font-weight: 300;
.font-normal	font-weight: 400;
.font-medium	font-weight: 500;
.font-semibold	font-weight: 600;
.font-bold	font-weight: 700;
.font-extrabold	font-weight: 800;
.font-black	font-weight: 900;

Uso basico

```
<p class="font-thin ...">El rapido zorro marron – font-thin</p>
<!-- ... cada grosor de 100 a 900 -->
```

Nota: el grosor renderizado depende de la fuente realmente cargada; se requiere una fuente variable o una familia con las nueve caras para ver cada paso.

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $font-weight-values: (
    "normal": 400,
    "medium": 500,
```

```

    "bold": 700
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();

```

Margin

Utilidades para controlar el margen de un elemento. Cada valor de \$spacing-map se emite para los siete prefijos: m-* (todos los lados), mx-*/my-* (horizontal/vertical) y mt-*/mr-*/mb-*/ml-* (un lado).

Referencia rapida

Escala	m-*	mx-*	my-*	mt-*	mr-*	mb-*	ml-*
0 -> 0	.m-0	.mx-0	.my-0	.mt-0	.mr-0	.mb-0	.ml-0
05 -> 0.125rem	.m-05	.mx-05	.my-05	.mt-05	.mr-05	.mb-05	.ml-05
base -> 1px	.m-base	.mx-base	.my-base	.mt-base	.mr-base	.mb-base	.ml-base
1 -> 0.25rem	.m-1	.mx-1	.my-1	.mt-1	.mr-1	.mb-1	.ml-1
1-5 -> 0.375rem	.m-1-5	.mx-1-5	.my-1-5	.mt-1-5	.mr-1-5	.mb-1-5	.ml-1-5
2 -> 0.5rem	.m-2	.mx-2	.my-2	.mt-2	.mr-2	.mb-2	.ml-2
2-5 -> 0.625rem	.m-2-5	.mx-2-5	.my-2-5	.mt-2-5	.mr-2-5	.mb-2-5	.ml-2-5
3 -> 0.75rem	.m-3	.mx-3	.my-3	.mt-3	.mr-3	.mb-3	.ml-3
3-5 -> 0.875rem	.m-3-5	.mx-3-5	.my-3-5	.mt-3-5	.mr-3-5	.mb-3-5	.ml-3-5
4 -> 1rem	.m-4	.mx-4	.my-4	.mt-4	.mr-4	.mb-4	.ml-4
5 ->	.m-5	.mx-5	.my-5	.mt-5	.mr-5	.mb-5	.ml-5

Escala	m-*	mx-*	my-*	mt-*	mr-*	mb-*	ml-*
1.25rem							
6 ->	.m-6	.mx-6	.my-6	.mt-6	.mr-6	.mb-6	.ml-6
1.5rem							
7 ->	.m-7	.mx-7	.my-7	.mt-7	.mr-7	.mb-7	.ml-7
1.75rem							
8 ->	.m-8	.mx-8	.my-8	.mt-8	.mr-8	.mb-8	.ml-8
2rem							
9 ->	.m-9	.mx-9	.my-9	.mt-9	.mr-9	.mb-9	.ml-9
2.25rem							
10 ->	.m-10	.mx-10	.my-10	.mt-10	.mr-10	.mb-10	.ml-10
2.5rem							
11 ->	.m-11	.mx-11	.my-11	.mt-11	.mr-11	.mb-11	.ml-11
2.75rem							
12 ->	.m-12	.mx-12	.my-12	.mt-12	.mr-12	.mb-12	.ml-12
3rem							
14 ->	.m-14	.mx-14	.my-14	.mt-14	.mr-14	.mb-14	.ml-14
3.5rem							
16 ->	.m-16	.mx-16	.my-16	.mt-16	.mr-16	.mb-16	.ml-16
4rem							
20 ->	.m-20	.mx-20	.my-20	.mt-20	.mr-20	.mb-20	.ml-20
5rem							
24 ->	.m-24	.mx-24	.my-24	.mt-24	.mr-24	.mb-24	.ml-24
6rem							
28 ->	.m-28	.mx-28	.my-28	.mt-28	.mr-28	.mb-28	.ml-28
7rem							
32 ->	.m-32	.mx-32	.my-32	.mt-32	.mr-32	.mb-32	.ml-32
8rem							
36 ->	.m-36	.mx-36	.my-36	.mt-36	.mr-36	.mb-36	.ml-36
9rem							
40 ->	.m-40	.mx-40	.my-40	.mt-40	.mr-40	.mb-40	.ml-40
10rem							
44 ->	.m-44	.mx-44	.my-44	.mt-44	.mr-44	.mb-44	.ml-44
11rem							
48 ->	.m-48	.mx-48	.my-48	.mt-48	.mr-48	.mb-48	.ml-48
12rem							

Escala	m-*	mx-*	my-*	mt-*	mr-*	mb-*	ml-*
52 -> 13rem	.m-52	.mx-52	.my-52	.mt-52	.mr-52	.mb-52	.ml-52
56 -> 14rem	.m-56	.mx-56	.my-56	.mt-56	.mr-56	.mb-56	.ml-56
60 -> 15rem	.m-60	.mx-60	.my-60	.mt-60	.mr-60	.mb-60	.ml-60
64 -> 16rem	.m-64	.mx-64	.my-64	.mt-64	.mr-64	.mb-64	.ml-64
72 -> 18rem	.m-72	.mx-72	.my-72	.mt-72	.mr-72	.mb-72	.ml-72
80 -> 20rem	.m-80	.mx-80	.my-80	.mt-80	.mr-80	.mb-80	.ml-80
96 -> 24rem	.m-96	.mx-96	.my-96	.mt-96	.mr-96	.mb-96	.ml-96

Uso basico

Anade margin a todos los lados de un elemento:

```
<div class="m-8 ..." >m-8</div>
```

Anadir margin horizontal y vertical

Controla el margin horizontal con mx-* y el margin vertical con my-*:

```
<div class="mx-12 my-4 ..." >mx-12 my-4</div>
```

Margin en un solo lado

Usa mt-*, mr-*, mb-* y ml-* para anadir margin a un solo lado:

```
<div class="mt-8 mr-4 mb-2 ml-6 ..." >mt-8 . mr-4 . mb-2 . ml-6</div>
```

Personalizar

Sobreescribe \$spacing-map antes de importar las utilidades para cambiar toda la escala a la vez:

```
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-map: (
    "0": 0,
    "1": 0.5rem,
```

```
    "2": 1rem,  
    "4": 2rem  
  )  
);
```

Posicion

Utilidades para controlar como se posiciona un elemento en el documento.

Referencia rapida

Clase	Propiedades
.static	position: static;
.fixed	position: fixed;
.absolute	position: absolute;
.relative	position: relative;
.sticky	position: sticky;

Uso basico

relative crea el contexto de posicionamiento; absolute elimina el elemento del flujo y lo ancla al ancestro posicionado mas cercano:

```
<div class="relative ...">  
  <span class="absolute" style="top: .5rem; right: .5rem">...</span>  
</div>
```

El framework no emite utilidades top/right/bottom/left, asi que combina absolute con tus propios desplazamientos.

Posicion fija

fixed ancla un elemento al viewport; permanece en su lugar cuando la pagina se desplaza:

```
<header class="fixed" style="top: 0; left: 0; right: 0">...</header>  
<main class="p-8"><!-- se desplaza debajo del header --></main>
```

Posicion sticky

sticky mantiene un elemento en el flujo hasta que alcanza el borde de desplazamiento, luego lo fija:

```
<nav class="sticky" style="top: 0">...</nav>
```

Usa la capa z-sticky de Z-Index cuando el elemento sticky necesite estar por encima de otro contenido.

Personalizar

Las clases de posicion se generan desde la lista \$position-values en el modulo mixins:

```
@use "katanakit-css/src/scss/mixins" as m with (  
  $position-values: (static, fixed, absolute, relative, sticky)  
);  
@use "katanakit-css/src/scss/utilities" as u;  
  
@include u.generate-layout-utilities();
```

Align Items

Utilidades para controlar como se posicionan los elementos flex en el eje transversal. Requiere un contenedor flex (.flex o .inline-flex).

Referencia rapida

Clase	Propiedades
.items-start	align-items: flex-start;
.items-center	align-items: center;
.items-end	align-items: flex-end;
.items-stretch	align-items: stretch;

Uso basico

Alinea los elementos en el eje transversal de un contenedor flex. El contenedor de demo tiene una altura fija para que la diferencia sea visible:

```
<div class="flex items-start gap-2 h-24">...</div>  
<div class="flex items-center gap-2 h-24">...</div>  
<div class="flex items-end gap-2 h-24">...</div>
```

Estirar

items-stretch (el valor por defecto de CSS) hace que cada elemento llene el eje transversal:

```
<div class="flex items-stretch h-24">  
  <div>...</div>
```

```
<div>...</div>
</div>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $align-items-map: (
    "start": flex-start,
    "center": center,
    "end": flex-end,
    "stretch": stretch
  )
);

@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-flex-utilities();
```

Color de borde

Utilidades para controlar el color de borde de un elemento, generadas por `v.border-utilities()` a partir de las paletas de color. Combinadas con las utilidades de Ancho de borde.

Referencia rapida

[illegible]

Paleta	100	200	300	400	500	600	700
	-	-	-	-	-	-	-
	danger-100	danger-200	danger-300	danger-400	danger-500	danger-600	danger-700
success	.border	.border	.border	.border	.border	.border	.border
	-	-	-	-	-	-	-
	success-100	success-200	success-300	success-400	success-500	success-600	success-700

Especial	Clase	Propiedades
black	.border-black	border-color: hsl(0, 0%, 3%);
white	.border-white	border-color: hsl(0, 0%, 98%);
transparent	.border-transparent	border-color: transparent;
current	.border-current	border-color: currentColor;

Uso basico

```
<div class="border-2 border-purple-500 ...">purple-500</div>
```

Usar currentColor

border-current hereda el color de texto del elemento, lo que mantiene los bordes y las etiquetas sincronizados:

```
<span class="border border-current text-purple-500 rounded-md p-2">badge</span>
```

Personalizar

```
@use "katanakit-css/src/scss/variables" as v;
```

```
// Solo las paletas danger y success:
```

```
@include v.border-utilities(("danger", "success"));
```

```
// Todas las paletas, sin colores especiales:
```

```
@include v.border-utilities($special: false);
```

```
// Todo:  
@include v.all-utilities();
```

Sombra de caja

Utilidades para controlar la sombra de caja de un elemento, generadas desde \$shadow-map. La clase base .shadow se emite automaticamente con el modulo utilities.

Referencia rapida

Clase	Propiedades
.shadow	box-shadow: 0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1);
.shadow-none	box-shadow: none;
.shadow-sm	box-shadow: 0 1px 2px 0 rgb(0 0 0 / 0.05);
.shadow-md	box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
.shadow-lg	box-shadow: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
.shadow-xl	box-shadow: 0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.1);
.shadow-2xl	box-shadow: 0 25px 50px -12px rgb(0 0 0 / 0.25);
.shadow-inner	box-shadow: inset 0 2px 4px 0 rgb(0 0 0 / 0.05);

Uso basico

```
<div class="shadow-md ...">shadow-md</div>  
<div class="shadow-inner ...">shadow-inner</div>
```

Eliminar sombras

shadow-none elimina una sombra existente; util para sobrescribir una sombra establecida por otra regla:

```
<div class="shadow-lg shadow-none ...">sin sombra</div>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $shadow-map: (
    "none": none,
    "sm": 0 1px 2px 0 rgb(0 0 0 / 0.05),
    "lg": 0 10px 15px -3px rgb(0 0 0 / 0.1)
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilitiies();
```

Gap

Utilidades para controlar los separadores entre elementos grid y flexbox, generadas desde \$spacing-map. La clase base .gap es gap: 1rem.

Referencia rapida

Escala	gap-*	gap-x-*	gap-y-*
0 -> 0	.gap-0	.gap-x-0	.gap-y-0
05 -> 0.125rem	.gap-05	.gap-x-05	.gap-y-05
base -> 1px	.gap-base	.gap-x-base	.gap-y-base
1 -> 0.25rem	.gap-1	.gap-x-1	.gap-y-1
1-5 -> 0.375rem	.gap-1-5	.gap-x-1-5	.gap-y-1-5
2 -> 0.5rem	.gap-2	.gap-x-2	.gap-y-2
2-5 -> 0.625rem	.gap-2-5	.gap-x-2-5	.gap-y-2-5
3 -> 0.75rem	.gap-3	.gap-x-3	.gap-y-3
3-5 -> 0.875rem	.gap-3-5	.gap-x-3-5	.gap-y-3-5
4 -> 1rem	.gap-4	.gap-x-4	.gap-y-4
5 -> 1.25rem	.gap-5	.gap-x-5	.gap-y-5
6 -> 1.5rem	.gap-6	.gap-x-6	.gap-y-6
7 -> 1.75rem	.gap-7	.gap-x-7	.gap-y-7
8 -> 2rem	.gap-8	.gap-x-8	.gap-y-8
9 -> 2.25rem	.gap-9	.gap-x-9	.gap-y-9

Escala	gap-*	gap-x-*	gap-y-*
10 -> 2.5rem	.gap-10	.gap-x-10	.gap-y-10
11 -> 2.75rem	.gap-11	.gap-x-11	.gap-y-11
12 -> 3rem	.gap-12	.gap-x-12	.gap-y-12
14 -> 3.5rem	.gap-14	.gap-x-14	.gap-y-14
16 -> 4rem	.gap-16	.gap-x-16	.gap-y-16
20 -> 5rem	.gap-20	.gap-x-20	.gap-y-20
24 -> 6rem	.gap-24	.gap-x-24	.gap-y-24
28 -> 7rem	.gap-28	.gap-x-28	.gap-y-28
32 -> 8rem	.gap-32	.gap-x-32	.gap-y-32
36 -> 9rem	.gap-36	.gap-x-36	.gap-y-36
40 -> 10rem	.gap-40	.gap-x-40	.gap-y-40
44 -> 11rem	.gap-44	.gap-x-44	.gap-y-44
48 -> 12rem	.gap-48	.gap-x-48	.gap-y-48
52 -> 13rem	.gap-52	.gap-x-52	.gap-y-52
56 -> 14rem	.gap-56	.gap-x-56	.gap-y-56
60 -> 15rem	.gap-60	.gap-x-60	.gap-y-60
64 -> 16rem	.gap-64	.gap-x-64	.gap-y-64
72 -> 18rem	.gap-72	.gap-x-72	.gap-y-72
80 -> 20rem	.gap-80	.gap-x-80	.gap-y-80
96 -> 24rem	.gap-96	.gap-x-96	.gap-y-96

Clase	Propiedades
.gap	gap: 1rem;

Uso basico

Añade separadores entre elementos flex:

```
<div class="flex gap-4">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

Usar gap de fila y columna

gap-x-* establece el separador de columna, gap-y-* el de fila:

```
<div class="grid gap-x-8 gap-y-2" style="grid-template-columns:
repeat(3, 1fr)">
  <!-- 6 celdas -->
</div>
```

Usar la clase gap base

.gap es un atajo para gap: 1rem (el mismo valor que .gap-4):

```
<div class="flex gap">...</div>
```

Personalizar

Sobreescribe \$spacing-map antes de importar las utilidades para cambiar toda la escala a la vez:

```
@use "katanakit-css/src/scss/variables" as v with (
  $spacing-map: (
    "0": 0,
    "1": 0.5rem,
    "2": 1rem,
    "4": 2rem
  )
);
```

Overflow

Utilidades para controlar como maneja un elemento el contenido que no cabe en su caja. Generadas desde la lista \$overflow-values.

Referencia rapida

Clase	Propiedades
.overflow-auto	overflow: auto;
.overflow-hidden	overflow: hidden;
.overflow-scroll	overflow: scroll;
.overflow-visible	overflow: visible;

Uso basico

Recorta el contenido que excede una altura o anchura fija:

```
<div class="w-64 h-24 overflow-hidden ..." >...</div>
<div class="w-64 h-24 overflow-auto ..." >...</div>
```

- overflow-hidden recorta sin desplazamiento.
- overflow-auto muestra barras de desplazamiento solo cuando se necesitan.
- overflow-scroll siempre reserva espacio para barras de desplazamiento.
- overflow-visible (el valor por defecto del navegador) deja que el contenido se desborde.

Combinar con utilidades de dimensiones

overflow-* se combina con las utilidades de Ancho y Alto para crear regiones desplazables:

```
<div class="max-h-64 overflow-auto">
  <!-- lista larga -->
</div>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $overflow-values: (auto, hidden, scroll, visible)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Alineacion de texto

Utilidades para controlar la alineacion del texto, generadas desde la lista \$text-align-values.

Referencia rapida

Clase	Propiedades
.text-left	text-align: left;
.text-center	text-align: center;
.text-right	text-align: right;
.text-justify	text-align: justify;

Uso basico

```
<p class="text-left ..." >...</p>
<p class="text-center ..." >...</p>
<p class="text-right ..." >...</p>
<p class="text-justify ..." >...</p>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $text-align-values: (left, center, right)
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-layout-utilities();
```

Justify Content

Utilidades para controlar como se distribuyen los elementos flex a lo largo del eje principal. Requiere un contenedor flex (.flex o .inline-flex).

Referencia rapida

Clase	Propiedades
.justify-start	justify-content: flex-start;
.justify-center	justify-content: center;
.justify-end	justify-content: flex-end;
.justify-between	justify-content: space-between;
.justify-around	justify-content: space-around;
.justify-evenly	justify-content: space-evenly;

Uso basico

```
<div class="flex justify-start ..." >...</div>
<div class="flex justify-center ..." >...</div>
<div class="flex justify-end ..." >...</div>
<div class="flex justify-between ..." >...</div>
<div class="flex justify-around ..." >...</div>
<div class="flex justify-evenly ..." >...</div>
```

justify-between empuja el primer elemento al inicio y el ultimo al final;
justify-around da a cada elemento medios espacios iguales; justify-evenly da a cada espacio el mismo tamaño.

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (  
  $justify-content-map: (  
    "start": flex-start,  
    "center": center,  
    "end": flex-end,  
    "between": space-between,  
    "around": space-around,  
    "evenly": space-evenly  
  )  
);  
@use "katanakit-css/src/scss/utilities" as u;  
  
@include u.generate-flex-utilities();
```

Opacidad

Utilidades para controlar la opacidad de un elemento, generadas desde el mapa \$opacity-values. A diferencia de hidden, un elemento con opacity-0 sigue ocupando espacio y recibe eventos de puntero.

Referencia rápida

Clase	Propiedades
.opacity-0	opacity: 0;
.opacity-5	opacity: 0.05;
.opacity-10	opacity: 0.1;
.opacity-20	opacity: 0.2;
.opacity-25	opacity: 0.25;
.opacity-30	opacity: 0.3;
.opacity-40	opacity: 0.4;
.opacity-50	opacity: 0.5;
.opacity-60	opacity: 0.6;
.opacity-70	opacity: 0.7;
.opacity-75	opacity: 0.75;
.opacity-80	opacity: 0.8;

Clase	Propiedades
.opacity-90	opacity: 0.9;
.opacity-95	opacity: 0.95;
.opacity-100	opacity: 1;

Uso basico

```
<div class="opacity-50 ...">50</div>
<div class="opacity-100 ...">100</div>
```

Combinar con transiciones

La opacidad se combina con las utilidades de Transiciones para efectos de desvanecimiento:

```
<button
  class="hover-bg-purple-500 bg-purple-300 duration-200 ease-out"
  style="transition-property: background-color"
>
  Desvanecer via hover-bg
</button>
```

Nota: no existe una variante .hover-opacity-* en este framework; las variantes hover de color son .hover-text-* y .hover-bg-* (consulta Color de texto y Color de fondo).

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $opacity-values: (
    "0": 0,
    "25": 0.25,
    "50": 0.5,
    "75": 0.75,
    "100": 1
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();
```

Espacio en blanco

Utilidades para controlar como se manejan los espacios en blanco dentro de un elemento y como se rompen las palabras largas. Generadas desde `$white-space-list` y `$overflow-wrap-list`.

Referencia rapida

Clase	Propiedades
<code>.whitespace-nowrap</code>	<code>white-space: nowrap;</code>
<code>.whitespace-pre</code>	<code>white-space: pre;</code>
<code>.whitespace-normal</code>	<code>white-space: normal;</code>
<code>.wrap-break-word</code>	<code>overflow-wrap: break-word;</code>

Uso basico

`whitespace-nowrap` evita que el texto se ajuste. Combinado con una utilidad `overflow-*` produce truncamiento de una sola linea:

```
<p class="whitespace-nowrap overflow-hidden ...">...</p>
<p class="whitespace-normal ...">...</p>
```

Preservar formato

`whitespace-pre` preserva cada espacio y salto de linea exactamente como esta escrito en el markup:

```
<pre class="whitespace-pre ...">linea uno
  linea dos (indentacion conservada)
linea tres</pre>
```

Romper palabras largas

`wrap-break-word` permite que las palabras largas sin puntos de corte (URLs, tokens) se ajusten en lugar de desbordarse:

```
<p class="wrap-break-word
w-64 ...">https://example.com/muy/larga/ruta</p>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $white-space-list: (nowrap, pre, normal),
  $overflow-wrap-list: (break-word,)
);
```

```
@use "katanakit-css/src/scss/utilities" as u;
```

```
@include u.generate-layout-utilities();
```

Z-Index

Utilidades para controlar el orden de apilamiento de un elemento, generadas desde \$z-layers-map. Las capas con nombre (dropdown ... tooltip) mantienen la consistencia de la UI superpuesta en todo el proyecto.

Referencia rapida

Clase	Propiedades
.z-auto	z-index: auto;
.z-0	z-index: 0;
.z-10	z-index: 10;
.z-20	z-index: 20;
.z-30	z-index: 30;
.z-40	z-index: 40;
.z-50	z-index: 50;
.z-dropdown	z-index: 1000;
.z-sticky	z-index: 1020;
.z-fixed	z-index: 1030;
.z-modal	z-index: 1040;
.z-popover	z-index: 1050;
.z-tooltip	z-index: 1060;

Uso basico

z-index solo se aplica a elementos posicionados; combinado con las utilidades de Posicion:

```
<div class="relative">
  <div class="absolute z-10 ...">z-10</div>
  <div class="absolute z-20 ...">z-20</div>
  <div class="absolute z-30 ...">z-30</div>
</div>
```

Usar capas con nombre

Las capas con nombre mapean a los roles semanticos en \$z-layers-map; usalas para que los modales siempre esten por encima de los dropdowns, los tooltips por encima de los popovers, y asi sucesivamente:

```
<header class="sticky z-sticky">...</header>
<nav class="absolute z-dropdown">...</nav>
<div class="fixed z-modal">...</div>
<div class="absolute z-tooltip">...</div>
```

Personalizar

Sobreescribe el mapa de capas en el modulo mixins:

```
@use "katanakit-css/src/scss/mixins" as m with (
  $z-layers-map: (
    "auto": auto,
    "0": 0,
    "10": 10,
    "dropdown": 100,
    "modal": 200,
    "tooltip": 300
  )
);
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utililities();
```

Transiciones

Utilidades para controlar transition-duration y transition-timing-function de un elemento, generadas desde \$transition-duration-map y \$transition-timing-map.

Estas utilidades establecen **solo** la duracion y la funcion de temporizacion. El framework no emite una clase .transition; declara transition-property tu mismo (por ejemplo transition: color), o usa las utilidades transition* dentro de @apply.

Referencia rapida

Clase	Propiedades
.duration-75	transition-duration: 75ms;

Clase	Propiedades
.duration-100	transition-duration: 100ms;
.duration-150	transition-duration: 150ms;
.duration-200	transition-duration: 200ms;
.duration-300	transition-duration: 300ms;
.duration-500	transition-duration: 500ms;
.duration-700	transition-duration: 700ms;
.duration-1000	transition-duration: 1000ms;

Clase	Propiedades
.ease-linear	transition-timing-function: linear;
.ease-in	transition-timing-function: cubic-bezier(0.4, 0, 1, 1);
.ease-out	transition-timing-function: cubic-bezier(0, 0, 0.2, 1);
.ease-in-out	transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);

Uso basico

Pasa el raton sobre el boton para ver la transicion en accion:

```
<button
  class="hover-bg-purple-500 bg-purple-300 duration-300 ease-out"
  style="transition-property: background-color"
>
  duration-300 ease-out
</button>
```

Personalizar

```
@use "katanakit-css/src/scss/mixins" as m with (
  $transition-duration-map: (
    "fast": 100ms,
    "normal": 300ms,
    "slow": 1000ms
  ),
  $transition-timing-map: (
    "linear": linear,
```

```

        "in-out": cubic-bezier(0.4, 0, 0.2, 1)
    )
};
@use "katanakit-css/src/scss/utilities" as u;

@include u.generate-effects-utilities();

```

Mixins de layout

Grid

Mixins de layout CSS Grid en el modulo mixins. Variables configurables: \$grid-gap-default: f.rem(10px) !default y \$grid-columns-default: 12 !default.

```
@use "katanakit-css/src/scss/mixins" as m;
```

Columnas responsivas

Mixin	Comportamiento
m.grid-responsive(\$min-size, \$mode: fill, \$max-size: 1fr, \$gap: null)	grid-template-columns: repeat(auto-fill auto-fit, minmax(min, max)). \$mode es fill o fit (error en caso contrario).
m.grid-autofill(\$min-size, \$max-size: 1fr, \$gap: null)	Atajo para fill.
m.grid-autofit(\$min-size, \$max-size: 1fr, \$gap: null)	Atajo para fit.
m.grid-fixed-columns(\$col-width, \$gap: null)	repeat(auto-fill, <width>) — ancho de columna fijo, sin minmax.
m.grid-breakpoint-columns(\$columns-map, \$gap: \$grid-gap-default, \$align-items: stretch)	Diferentes conteos de columnas por breakpoint. \$columns-map usa una clave default (usada por debajo de sm) mas una clave por breakpoint.

```
@use "katanakit-css/src/scss/mixins" as m;
```

```

.cards { @include m.grid-autofill(240px, 1fr, 1rem); }

.dashboard {
    @include m.grid-breakpoint-columns(("default": 1, "md": 2, "xl": 4),
    1rem);
}

```

Contenedores de layout

Mixin	Comportamiento
<code>m.grid-container(\$cols: 12, \$rows: null, \$gap: \$grid-gap-default, \$place-items: center, \$place-content: center, \$align-items: null, \$justify-content: null)</code>	Columnas 1fr iguales. Usa <code>place-items/place-content</code> , o <code>align-items/justify-content</code> cuando los parametros <code>place-*</code> son null.
<code>m.grid-center(\$gap: null)</code>	Atajo de centrado (<code>place-items/place-content: center</code>).
<code>m.grid-gap(\$value: \$grid-gap-default, \$var-name: --grid-gap)</code>	Establece una propiedad personalizada y <code>gap: var(--grid-gap)</code> .
<code>m.grid-areas(\$areas, \$gap: null)</code>	<code>grid-template-areas</code> con la base de display.
<code>m.grid-area(\$name)</code>	Asigna un elemento a un area de plantilla.
<code>m.subgrid(\$axis: both)</code>	<code>grid-template-columns/rows: subgrid</code> para <code>column/row/both</code> .
<code>m.grid-masonry(\$axis: block, \$gap: null)</code>	Experimental masonry via <code>grid-template-rows: masonry</code> (el soporte del navegador no es universal). <code>\$axis: inline</code> mueve el eje masonry a columnas.

`@use "katanakit-css/src/scss/mixins" as m;`

```
.page {  
  @include m.grid-areas(  
    "header header"  
    "sidebar main"  
    "footer footer",  
    1rem  
  );  
  
  .header { @include m.grid-area("header"); }  
  .sidebar { @include m.grid-area("sidebar"); }  
  .main    { @include m.grid-area("main"); }
```

```
.footer { @include m.grid-area("footer"); }
}
```

Elementos de grid

Mixin	Comportamiento
m.grid-span(\$cols: 1, \$rows: null)	grid-column: span N (+ grid-row: span N cuando se establece \$rows).
m.grid-placement(\$col-start: 1, \$col-end: -1, \$row-start: null, \$row-end: null)	grid-column: start / end y, cuando se proporcionan ambas filas, grid-row.
m.grid-item-center	place-self: center center.
m.grid-item-full(\$col: "1 / -1", \$row: null)	Ocupa todas las columnas (personalizable via \$col).
@use "katanakit-css/src/scss/mixins" as m;	
<pre>main { @include m.grid-placement(2, 4, 1, 2); } // grid-column: 2/4; grid-row: 1/2 header { @include m.grid-item-full(); } // grid-column: 1 / -1 .card { @include m.grid-span(3); }</pre>	

Apilar elementos

Mixin	Comportamiento
m.grid-stack(\$dp: grid, \$parent: null, \$children: ("*"), \$col: 1, \$row: 1)	Convierte el elemento actual (o el .parent nombrado) en un grid y coloca cada selector \$child en la misma celda en (\$col, \$row), de modo que los hijos se superponen.
m.grid-stack-item(\$col: 1, \$row: 1)	Coloca el elemento actual en una celda de apilamiento.
@use "katanakit-css/src/scss/mixins" as m;	

```
.stack { @include m.grid-stack(grid, null, ("span", "div"), 2, 1); }
// .stack { display: grid }
// .stack > span, .stack > div { grid-column: 2 / 3; grid-row: 1 /
2; }
```

grid-stack requiere numeros sin unidad >= 1 y acepta \$parent con o sin punto inicial.

Ejemplo de dashboard

Un layout de dashboard completo que combina contenedores, spans y breakpoints:

```
@use "katanakit-css/src/scss/mixins" as m;
@use "katanakit-css/src/scss/variables" as v;

.dashboard {
  @include m.grid-container(12, $gap: 1rem, $place-items: stretch,
    $place-content: stretch);

  .sidebar {
    @include m.grid-span(3);
  }

  .content {
    @include m.grid-span(9);
  }

  @include m.md-down {
    grid-template-columns: 1fr;

    .sidebar,
    .content {
      @include m.grid-span(1);
    }
  }
}

.cards { @include m.grid-responsive(280px, fill, 1fr, 1.5rem); }

.pinned-card { @include m.grid-placement(2, 4, 1, 2); }

.modal-overlay {
  @include m.grid-center(0);

  .modal {
    @include m.grid-item-center;
    max-width: v.container("md");
  }
}
```

Consulta Ejemplos para el código fuente completo de `grid-dashboard.scss`.

Flexbox

Mixins de layout Flexbox en el módulo mixins.

```
@use "katanakit-css/src/scss/mixins" as m;
```

Contenedores

Mixin	Comportamiento
<code>m.flex-container(\$direction: row, \$wrap: nowrap, \$gap: null, \$align-items: stretch, \$justify-content: flex-start, \$align-content: normal, \$place-items: null, \$place-content: null)</code>	Display flex con dirección/envoltura. <code>place-items</code> tiene prioridad sobre <code>align-items</code> ; <code>place-content</code> tiene prioridad sobre <code>justify-content</code> / <code>align-content</code> .
<code>m.flex-center(\$gap: null)</code>	Atajo de centrado (<code>place-items/place-content: center</code>).
<code>m.flex-gap(\$value: 1rem, \$var-name: --flex-gap)</code>	Establece una propiedad personalizada y <code>gap: var(--flex-gap)</code> .

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.toolbar { @include m.flex-container(row, nowrap, 0.5rem, center, space-between); }  
.hero     { @include m.flex-center(2rem); }  
.list     { @include m.flex-gap(1.5rem); }
```

Dimensiones de elementos

Mixin	Comportamiento
<code>m.flex-grow(\$grow: 1)</code>	<code>flex-grow</code> .
<code>m.flex-shrink(\$shrink: 1)</code>	<code>flex-shrink</code> .
<code>m.flex-basis(\$basis: auto)</code>	<code>flex-basis</code> .
<code>m.flex-item(\$grow: 1, \$shrink: 1, \$basis: auto)</code>	<code>flex: <grow> <shrink> <basis></code> .
<code>m.flex-item-center</code>	<code>align-self: center</code> .
<code>m.flex-item-full</code>	<code>flex: 1 1 100%</code> .

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.logo { @include m.flex-item(0, 0, auto); } // nunca crece, nunca se
encoge
.search { @include m.flex-item-full(); } // llena el espacio
restante
.badge { @include m.flex-item-center; }
```

Ejemplo: barra de navegacion

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
.navbar {
  @include m.flex-container(row, nowrap, 1rem, center, space-between);

  .brand {
    @include m.flex-item(0, 0, auto);
  }

  .spacer {
    @include m.flex-item(1, 1, auto); // empuja el resto a los bordes
  }

  .actions {
    @include m.flex-container(row, nowrap, 0.5rem, center, flex-end);
  }
}
```

Consulta las utilidades Flexbox para los equivalentes basados en clases (.flex, .items-*, .justify-*, .gap-*).

Breakpoints

Referencia tecnica del motor de mixins responsivos en el modulo mixins. Para una vista conceptual y las consultas de características, consulta Breakpoints.

```
@use "katanakit-css/src/scss/mixins" as m;
```

Configuracion

m.\$breakpoints (todos !default):

Nivel	Ancho
xs	0
sm	640px
md	768px

Nivel	Ancho
lg	1024px
x1	1280px
2x1	1536px
3x1	1920px

```
breakpoint($from, $direction: up, $to: null)
```

Abre una consulta @media. \$from y \$to aceptan un nombre de nivel (string) o un numero px en bruto (los numeros sin unidad se tratan como px). bp(\$args...) es un alias que reenvia sus argumentos.

Direccion	Consulta media
up	(min-width: X)
down	(max-width: calc(X - 0.02px))
only	(min-width: X) and (max-width: calc(Y - 0.02px)) — requiere \$to
between	(min-width: X) and (max-width: Y) — requiere \$to

```
@include m.breakpoint("sm", up)      { /* >= 640px */ }
@include m.breakpoint("md", down)    { /* <= 767.98px */ }
@include m.breakpoint("sm", only, "md") { /* 640px-767.98px */ }
@include m.breakpoint("md", between, "x1") { /* 768px-1280px */ }
@include m.breakpoint(500px, up)      { /* valor en bruto */ }
@include m.bp("lg", down)             { /* alias */ }
```

Una direccion invalida o un \$to faltante lanza un @error en tiempo de compilacion.

Mixins con nombre — direccion up

Mixin	Equivalente
m.xs	breakpoint("xs", up)
m.sm	breakpoint("sm", up)
m.md	breakpoint("md", up)
m.lg	breakpoint("lg", up)
m.x1	breakpoint("x1", up)
m.xx1	breakpoint("2x1", up)
m.xxx1	breakpoint("3x1", up)

```
.sidebar {
  display: none;

  @include m.lg { display: block; }
}
```

Mixins con nombre — direccion down

Mixin	Equivalente
m.xs-down	breakpoint("xs", down)
m.sm-down	breakpoint("sm", down)
m.md-down	breakpoint("md", down)
m.lg-down	breakpoint("lg", down)
m.xl-down	breakpoint("xl", down)
m.x2l-down	breakpoint("2xl", down)
m.x3l-down	breakpoint("3xl", down)
m.xx1-down	alias de m.x2l-down
m.xxx1-down	alias de m.x3l-down

```
.grid {
  grid-template-columns: repeat(4, 1fr);

  @include m.md-down { grid-template-columns: 1fr; }
  @include m.xx1-down { padding-inline: 2rem; }
}
```

Mixins de consultas de características

Mixin	Consulta media
m.portrait	(orientation: portrait)
m.landscape	(orientation: landscape)
m.reduced-motion	(prefers-reduced-motion: reduce)
m.hoverable	(hover: hover) and (pointer: fine)
m.touch	(hover: none) and (pointer: coarse)
m.dark-mode	(prefers-color-scheme: dark)
m.light-mode	(prefers-color-scheme: light)

```
.menu {
  @include m.touch { padding: 1rem; } // blancos mas grandes en
  dispositivos tactiles
  @include m.reduced-motion { transition: none; }
}
```

Referencia

Funciones

Funciones puras auxiliares en el modulo functions (src/scss/_functions.scss). Todas lanzan @error en tiempo de compilacion cuando la entrada es invalida en lugar de fallar silenciosamente.

```
@use "katanakit-css/src/scss/functions" as f;
```

\$base-font-size: 16px !default es la linea base en px usada por rem()/px().

Conversion de unidades

Firma	Comportamiento
f.rem(\$size)	Convierte px o numeros sin unidad a rem usando \$base-font-size. f.rem(16px) y f.rem(16) ambos devuelven 1rem.
f.px(\$size)	Convierte rem o numeros sin unidad a px. f.px(1) devuelve 16px, f.px(1.5rem) devuelve 24px.
f.to-unit(\$value, \$unit: "rem")	Añade una unidad (rem, px, em o %) a un numero sin unidad. Los valores que ya tienen unidad se devuelven sin cambios. Lanza @error con una unidad invalida.
f.strip-unit(\$value)	Elimina la unidad: f.strip-unit(16px) devuelve 16.

```
@use "katanakit-css/src/scss/functions" as f;
```

```
.card {
  padding: f.rem(24); // 1.5rem
  margin: f.px(1); // 16px
}
```

```
width: f.to-unit(50, "%"); // 50%
}
```

Tipografia fluida

Firma	Comportamiento
<code>f.fluid(\$min, \$max, \$min-vw: 320px, \$max-vw: 1920px)</code>	Devuelve una expresion <code>clamp()</code> que interpola <code>\$min</code> -> <code>\$max</code> a traves de <code>\$min-vw</code> -> <code>\$max-vw</code> . Las entradas sin unidad se tratan como px.
<code>font-size: f.fluid(16px, 24px);</code> <code>// clamp(16px, 0.5vw + 14.4px, 24px)</code>	
<code>font-size: f.fluid(1rem, 2rem, 768px, 1280px);</code> <code>// clamp(1rem, 0.1953125vw - 0.5px, 2rem)</code>	
<code>h1 { font-size: f.fluid(2rem, 4rem); }</code>	

Helpers de color

Firma	Comportamiento
<code>f.tint(\$color, \$amount: 10%)</code>	Mezcla <code>\$color</code> con blanco.
<code>f.shade(\$color, \$amount: 10%)</code>	Mezcla <code>\$color</code> con negro.
<code>f.saturate-color(\$color, \$amount: 10%)</code>	Aumenta la saturacion via <code>color.scale</code> .
<code>f.desaturate-color(\$color, \$amount: 10%)</code>	Reduce la saturacion via <code>color.scale</code> .
<code>f.complement(\$color)</code>	Devuelve el complemento de tono.
<code>f.contrast(\$color, \$light: white, \$dark: black)</code>	Elige un color de texto legible a partir del brillo percibido de <code>\$color</code> : los colores brillantes devuelven <code>\$dark</code> , los oscuros devuelven <code>\$light</code> .
<code>f.color-mix-var(\$var-name, \$amount: 10%, \$mix-with: white)</code>	Emite <code>color-mix(in srgb, var(--name) <pct>, <mix-with>)</code> . Acepta un nombre de variable desnudo (<code>--info-500</code>) o una expresion <code>var(...)</code> completa.
<code>f.to-class(\$key)</code>	Escapa una clave de mapa para usar en un selector de clase: <code>.</code> -> <code>\.</code> , <code>/</code> -> <code>\/</code> , <code>%</code> -> <code>\%</code> .

```

@use "katanakit-css/src/scss/functions" as f;

.card {
  background: f.tint(#7d2bd4, 20%);
  border-color: f.shade(#7d2bd4, 15%);
  color: f.contrast(#7d2bd4); // blanco – el color es oscuro
}

.alert {
  background: f.color-mix-var("--danger-500", 10%);
  // background: color-mix(in srgb, var(--danger-500) 90%, white);
}

.surface {
  color: f.desaturate-color(#066bf9, 30%);
}

```

Todas las funciones de color lanzan un @error cuando reciben un no-color.

Referencia API

Referencia completa de katanakit-css **0.1.0** (no publicado). Cada firma, valor por defecto y clave de mapa en este documento fue verificado contra el código fuente en `src/scss/`. Este archivo es solo documentación; nunca modifica el código.

Namespaces de módulos

Todos los módulos se consumen con @use en minúsculas y un namespace explícito:

```

@use "katanakit-css/src/scss/functions" as f;
@use "katanakit-css/src/scss/variables" as v;
@use "katanakit-css/src/scss/mixins" as m;
@use "katanakit-css/src/scss/utilities" as u;

```

Cuando los módulos forman parte de tu propio árbol de código fuente, las rutas se vuelven relativas, por ejemplo `@use "functions" as f;`. La entrada pública `src/scss/main.scss` carga `reset`, `variables`, `functions`, `mixins` y `utilities`, luego llama:

```

@include v.generate-css-tokens();
@include v.generate-css-vars();

```



```
@include v.all-utilities();
@include u.generate-all-utilities();
```

Funciones (as f)

Fuente: `_functions.scss`.

Conversion de unidades

Firma	Comportamiento
<code>rem(\$size)</code>	Convierte px o numeros sin unidad a rem usando <code>\$base-font-size</code> (por defecto 16px). <code>rem(16px)</code> y <code>rem(16)</code> ambos devuelven 1rem.
<code>px(\$size)</code>	Convierte rem o numeros sin unidad a px. <code>px(1)</code> devuelve 16px, <code>px(1.5rem)</code> devuelve 24px.
<code>to-unit(\$value, \$unit: "rem")</code>	Añade una unidad (rem, px, em o %) a un numero ya sin unidad. Los valores que llevan unidad se devuelven sin cambios. Lanza <code>@error</code> con una unidad invalida.
<code>strip-unit(\$value)</code>	Elimina la unidad, por ejemplo <code>strip-unit(16px)</code> devuelve 16.

`$base-font-size: 16px` !default es el valor por defecto del navegador. Solo sobreescribelo si tu proyecto cambia el tamaño de fuente raíz (es una línea base en px, no una línea base “rem” de 10px).

Tipografía fluida

Firma	Comportamiento
<code>fluid(\$min, \$max, \$min-vw: 320px, \$max-vw: 1920px)</code>	Devuelve una expresión <code>clamp()</code> que interpola <code>\$min</code> -> <code>\$max</code> a través de <code>\$min-vw</code> -> <code>\$max-vw</code> . Las entradas sin unidad se tratan como px.

```
font-size: f.fluid(16px, 24px);
// clamp(16px, 0.5vw + 14.4px, 24px)
```

```
font-size: f.fluid(1rem, 2rem, 768px, 1280px);  
// clamp(1rem, 0.1953125vw - 0.5px, 2rem)
```

Helpers de color

Firma	Comportamiento
tint(\$color, \$amount: 10%)	Mezcla \$color con blanco.
shade(\$color, \$amount: 10%)	Mezcla \$color con negro.
saturate-color(\$color, \$amount: 10%)	Aumenta la saturacion via color.scale.
desaturate-color(\$color, \$amount: 10%)	Reduce la saturacion via color.scale.
complement(\$color)	Devuelve el complemento de tono.
contrast(\$color, \$light: white, \$dark: black)	Elige un color de texto legible a partir del brillo percibido de \$color: los colores brillantes devuelven \$dark, los oscuros devuelven \$light.
color-mix-var(\$var-name, \$amount: 10%, \$mix-with: white)	Emite color-mix(in srgb, var(--name) <pct>, <mix-with>). Acepta un nombre de variable desnudo ("--info-500") o una expresion var(...) completa.
to-class(\$key)	Escapa una clave de mapa para usar en un selector de clase. . se convierte en \., / en \/, % en \%. to-class("1-5") devuelve el string 1-5 (los puntos/guiones de las claves mismas se usan literalmente por los generadores).

Todas las funciones de color que reciben un no-color lanzan un @error.

Variables / tokens (as v)

Fuente: _variables.scss.

Mapas de tokens (todos !default)

\$font-families

Clave	Valor
sans-serif	ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif
serif	"Times New Roman", Trebuchet, Geneva, serif
mono	monospace

\$shadow-sizes

Clave	Valor
default	0 1px 3px 0 rgb(0 0 0 / 0.1), 0 1px 2px -1px rgb(0 0 0 / 0.1)
sm	0 1px 2px 0 rgb(0 0 0 / 0.05)
md	0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1)
lg	0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1)
xl	0 20px 25px -5px rgb(0 0 0 / 0.1), 0 8px 10px -6px rgb(0 0 0 / 0.1)
2xl	0 25px 50px -12px rgb(0 0 0 / 0.25)
inner	inset 0 2px 4px 0 rgb(0 0 0 / 0.05)

\$container-sizes — sm 24rem . md 28rem . lg 32rem . xl 36rem . 2xl 42rem . 3xl 48rem . 4xl 56rem . 5xl 64rem . 6xl 72rem.

\$breakpoint-sizes — xs 0 . sm 640px . md 768px . lg 1024px . xl 1280px . 2xl 1536px . 3xl 1920px.

\$spacing-scale

0 0 . 1 0.25rem . 2 0.5rem . 3 0.75rem . 4 1rem . 5 1.25rem . 6 1.5rem . 8 2rem . 10 2.5rem . 12 3rem . 16 4rem . 20 5rem . 24 6rem . 32 8rem.

\$radius

default 0.25rem . sm 0.125rem . md 0.375rem . lg 0.5rem . xl 0.75rem . 2xl 1rem . 3xl 1.5rem . full 9999px.

\$z-layers

auto auto . 0 0 . 10 10 . 20 20 . 30 30 . 40 40 . 50 50 . dropdown 1000 . sticky 1020 . fixed 1030 . modal 1040 . popover 1050 . tooltip 1060.

\$transition-durations — 75 75ms . 100 100ms . 150 150ms . 200 200ms . 300 300ms . 500 500ms . 700 700ms . 1000 1000ms.

\$transition-easings

linear linear . in cubic-bezier(0.4, 0, 1, 1) . out cubic-bezier(0, 0, 0.2, 1) . in-out cubic-bezier(0.4, 0, 0.2, 1).

Funciones de acceso

Firma	Devuelve	Notas
font-family(\$name)	lista de fuentes	sans-serif / serif / mono.
shadow(\$size: "default")	lista/valor de sombra	default ... inner.
container(\$size)	longitud	sm ... 6xl.
breakpoint(\$name)	longitud en px	xs ... 3xl.
spacing(\$size)	longitud	Acepta un numero o una clave string (spacing(4) = spacing("4")).
radius(\$size: "default")	longitud	default ... full.
z(\$layer)	numero/auto	Acepta un numero o un string (z(50) = z("50"), mas capas con nombre).
duration(\$ms)	duracion	Acepta un numero o string (duration(200) = duration("200")).
ease(\$name: "in-out")	funcion de temporizacion	linear / in / out / in-out.

Cada accesador lanza un @error en tiempo de compilacion que enumera las claves validas cuando la clave solicitada falta.

```
@mixin generate-css-tokens(...)
@mixin generate-css-tokens(
  $fonts: true, $shadows: true, $containers: true, $spacing: true,
  $radius: true, $z: true, $durations: true, $easings: true
)
```

Emite las familias de tokens como propiedades personalizadas en `:root`. Pasa `false` para omitir una familia.

```
:root {
  --font-sans-serif: ui-sans-serif, system-ui, ...;
  --shadow-md: ...;
  --container-xl: 36rem;
  --spacing-4: 1rem;
  --z-tooltip: 1060;
  --duration-200: 200ms;
  --ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
}
```

Los nombres de las variables no llevan prefijo de marca: son exactamente `--font-*`, `--shadow-*`, `--container-*`, `--spacing-*`, `--radius-*`, `--z-*`, `--duration-*` y `--ease-*`.

Colores (as v)

Fuente: `_variables.scss` (fusionado del antiguo `_colors.scss`).

Paletas

`$colors` contiene seis paletas con **siete tonos cada una** (100 mas claro -> 700 mas oscuro), almacenadas como `hsla(...)` y emitidas como `hsl(...)`:

Paleta	100	200	300	400	500	600	700
neutral	0 0% 93%	0 0% 86%	0 0% 71%	0 0% 50%	0 0% 35%	0 0% 24%	0 0% 12%
purple	269 60% 93%	269 70% 86%	269 70% 71%	269 66% 50%	269 66% 35%	269 60% 24%	269 50% 12%
info	215 95% 93%	215 95% 86%	215 95% 71%	215 95% 50%	215 95% 35%	215 95% 24%	215 95% 12%
warning	49 100% 93%	49 100% 86%	49 100% 71%	49 100% 50%	49 100% 35%	49 90% 24%	49 70% 12%

Paleta	100	200	300	400	500	600	700
danger	3 95%	3 95%	3 95%	3 95%	3 95%	3 95%	3 95%
	93%	86%	71%	50%	35%	24%	12%
success	147 95%	147 95%	147 95%	147 95%	147 95%	147 95%	147 95%
	93%	86%	71%	50%	35%	24%	12%

Los valores de las celdas anteriores son `hsl(tono, saturacion, luminosidad)`. No hay paletas adicionales y no hay nombres de color específicos de marca conectados al framework (consulta `colors-palette.md` para la paleta de marca personal, que es material de referencia solamente).

\$special-colors — `black hsl(0, 0%, 3%).white hsl(0, 0%, 98%).transparent transparent.current currentColor.`

\$shades: (100, 200, 300, 400, 500, 600, 700) !default — orden de iteración para funciones impulsadas por tonos.

Funciones de acceso

Firma	Devuelve
<code>get-color(\$name, \$shade: 300, \$alpha: 1)</code>	Un tono de paleta (o un color especial cuando <code>\$name</code> es especial). Aplica alfa cuando <code>\$alpha != 1</code> .
<code>get(\$name, \$alpha: 1)</code>	Abreviatura: <code>get-color(\$name, 500, \$alpha)</code> .
<code>alpha(\$name, \$shade: 500, \$alpha: 0.5)</code>	Abreviatura para variantes transparentes: <code>alpha("info", 300, 0.5) -> hsla(215, 95%, 71%, 0.5)</code> .
<pre>color: v.get-color("neutral", 500); // hsl(0, 0%, 35%) color: v.get("info"); // tono 500 border: 1px solid v.alpha("warning", 400, 0.25);</pre>	
<pre>@mixin generate-css-vars(\$root: ":root", \$palettes: null, \$include-special: true)</pre>	

Emite una propiedad personalizada por tono mas los colores especiales:

```
:root {
  --neutral-100: hsl(0, 0%, 93%);
  /* ... cada paleta y tono ... */
  --info-500: hsl(215, 95%, 35%);
  --white: hsl(0, 0%, 98%);
```

```
--black: hsl(0, 0%, 3%);
--transparent: transparent;
--current: currentColor;
}
```

\$palettes acepta:

- null — todas las paletas;
- un nombre de paleta o lista de nombres — solo esas paletas (por ejemplo "info", (neutral, success));
- un **mapa** — emitido tal cual. Asi es como theme() alimenta las paletas invertidas a un selector raiz personalizado.

Mixins de utilidad

Mixin	Clases
text-utilities(\$palettes: null, \$special: true)	.text-{palette}-{shade}, .text-{special} (por ejemplo .text-white, .text-neutral-500, .text-info-400) — propiedad color.
bg-utilities(\$palettes: null, \$special: true)	.bg-{palette}-{shade}, .bg-{special} — propiedad background-color.
border-utilities(\$palettes: null, \$special: true)	.border-{palette}-{shade}, .border-{special} — propiedad border-color.
hover-utilities(\$palettes: null, \$special: true)	.hover-text-{palette}-{shade}:hover, .hover-bg-{palette}-{shade}:hover, mas .hover-text-{special}:hover / .hover-bg-{special}:hover.
all-utilities(\$palettes: null, \$special: true)	Llama los cuatro mixins anteriores.

Las clases de utilidad usan **valores literales de color**, no referencias var(). Las propiedades personalizadas emitidas por generate-css-vars() estan ahi para tus propias reglas.

@mixin theme(\$mode, \$overrides: ())

Emita las paletas bajo :root[data-theme="#{\$mode}"] (sin colores especiales). Cuando \$mode: "dark", cada paleta se **invierte** para que la clave del tono mas

claro contenga el color mas oscuro y viceversa: tono 100 <-> 700, 200 <-> 600, 300 <-> 500. \$overrides se fusiona sobre el mapa resultante.

Tema (as v)

Fuente: `_variables.scss` (fusionado del antiguo `_theme.scss`).

Firma	Comportamiento
<code>theme(\$mode: "dark", \$overrides: ())</code>	Hook delgado que delega en <code>v.theme()</code> .
<code>@include v.theme("dark");</code> // <i>invierte paletas, emite :root[data-theme="dark"]</i>	

Breakpoints (as m)

Fuente: `_mixins.scss` (fusionado del antiguo `_breakpoints.scss`).

\$breakpoints(todos !default)

xs 0 . sm 640px . md 768px . lg 1024px . xl 1280px . 2xl 1536px . 3xl 1920px.

Las claves que contienen digitos (2xl, 3xl) son nombres de primera clase; pasalas como strings.

Mixin generico

Firma	Comportamiento
<code>breakpoint(\$from, \$direction: up, \$to: null)</code>	Abre una consulta @media. Acepta una clave o un numero px en bruto.
<code>bp(\$args...)</code>	Alias que reenvia argumentos a <code>breakpoint()</code> .

Direcciones:

- up — (min-width: X)
- down — (max-width: calc(X - 0.02px))
- only — (min-width: X) and (max-width: calc(Y - 0.02px)), requiere \$to
- between — (min-width: X) and (max-width: Y), requiere \$to


```
@use "katanakit-css/src/scss/mixins" as m;
```

```
@include m.breakpoint("lg", down) { ... }
```

```
@include m.bp("md", up) { ... } // el alias reenvia a breakpoint("md", up)
```

Mixins con nombre

up: xs, sm, md, lg, xl, xxl (alias para la clave 2xl), xxxl (alias para la clave 3xl).

down: xs-down, sm-down, md-down, lg-down, xl-down, x2l-down / x3l-down, mas los alias canonicos xxl-down (= x2l-down) y xxxl-down (= x3l-down).

```
@include m.xxl { ... } // @media (min-width: 1536px)
```

```
@include m.xxxl { ... } // @media (min-width: 1920px)
```

```
@include m.x2l-down { ... } // @media (max-width: calc(1536px - 0.02px))
```

Consultas de características

```
portrait.landscape.reduced-motion.hoverable((hover: hover) and  
(pointer: fine)).touch((hover: none) and (pointer: coarse)).dark-mode.  
light-mode.
```

Grid (as m)

Fuente: `_mixins.scss` (fusionado del antiguo `_grid.scss`).

Variables configurables: `$grid-gap-default: f.rem(10px) !default` y `$grid-columns-default: 12 !default`.

Columnas responsivas

Mixin	Comportamiento
<code>grid-responsive(\$min-size, \$mode: fill, \$max-size: 1fr, \$gap: null)</code>	<code>grid-template-columns: repeat(auto-fill auto-fit, minmax(min, max)).</code> \$mode es fill o fit (error en caso contrario).
<code>grid-autofill(\$min-size, \$max-size: 1fr, \$gap: null)</code>	Atajo para fill.
<code>grid-autofit(\$min-size, \$max-size: 1fr, \$gap: null)</code>	Atajo para fit.
<code>grid-fixed-columns(\$col-width,</code>	<code>repeat(auto-fill, <width>).</code>

Mixin	Comportamiento
<code>\$gap: null)</code>	
<code>grid-breakpoint-columns(\$columns-map, \$gap: \$grid-gap-default, \$align-items: stretch)</code>	Diferentes conteos de columnas por breakpoint. <code>\$columns-map</code> usa una clave default (usada por debajo de sm) y una clave por breakpoint.
<code>@use "katanakit-css/src/scss/mixins" as m;</code>	
<code>.cards { @include m.grid-autofill(240px, 1fr, 1rem); }</code>	
<code>.dashboard { @include m.grid-breakpoint-columns(("default": 1, "md": 2, "xl": 4), 1rem); }</code>	

Contenedores de layout

Mixin	Comportamiento
<code>grid-container(\$cols: 12, \$rows: null, \$gap: \$grid-gap-default, \$place-items: center, \$place-content: center, \$align-items: null, \$justify-content: null)</code>	Columnas 1fr iguales; place-items/place-content, o align-items/justify-content cuando los parametros place-* son null.
<code>grid-center(\$gap: null)</code>	Atajo de centrado (place-items/place-content: center).
<code>grid-gap(\$value: \$grid-gap-default, \$var-name: --grid-gap)</code>	Establece una propiedad personalizada y gap: var(--grid-gap).
<code>grid-areas(\$areas, \$gap: null)</code>	grid-template-areas con una base de display.
<code>grid-area(\$name)</code>	Asigna un elemento a un area de plantilla.
<code>grid-masonry(\$axis: block, \$gap: null)</code>	Experimental masonry via grid-template-rows: masonry (el soporte del navegador no es universal). <code>\$axis: inline</code> mueve el eje masonry a columnas.

Elementos de grid

Mixin	Comportamiento
<code>grid-span(\$cols: 1, \$rows: null)</code>	<code>grid-column: span N(+ grid-row: span N).</code>
<code>grid-placement(\$col-start: 1, \$col-end: -1, \$row-start: null, \$row-end: null)</code>	<code>grid-column: start / end</code> y, cuando se proporcionan, <code>grid-row</code> .
<code>grid-item-center</code>	<code>place-self: center center.</code>
<code>grid-item-full(\$col: "1 / -1", \$row: null)</code>	Ocupa todas las columnas (personalizable con el string <code>\$col</code>).
<pre>main { @include m.grid-placement(2, 4, 1, 2); } // grid-column: 2/4; grid-row: 1/2 header { @include m.grid-item-full(); } // grid-column: 1 / -1</pre>	

Apilamiento

Mixin	Comportamiento
<code>grid-stack(\$dp: grid, \$parent: null, \$children: ("*"), \$col: 1, \$row: 1)</code>	Convierte el elemento actual (o un <code>.parent</code> nombrado) en un grid y coloca cada selector <code>\$child</code> en la misma celda en (<code>\$col</code> , <code>\$row</code>) para que los hijos se superpongan.
<code>grid-stack-item(\$col: 1, \$row: 1)</code>	Coloca el elemento actual en una celda de apilamiento.
<pre>.stack { @include m.grid-stack(grid, null, ("span", "div"), 2, 1); } // .stack { display: grid } y .stack > span, .stack > div colocados en 2/3 x 1/2</pre>	
<code>grid-stack</code> requiere numeros sin unidad ≥ 1 y acepta <code>\$parent</code> con o sin punto inicial.	

Flex (as m)

Fuente: `_mixins.scss` (fusionado del antiguo `_flex.scss`).

Mixin	Comportamiento
<code>flex-container(\$direction: row, \$wrap: nowrap, \$gap: null,</code>	<code>Display flex</code> con direccion/envoltura; <code>place-items</code> tiene prioridad sobre

Mixin	Comportamiento
<code>\$align-items: stretch, \$justify-content: flex-start, \$align-content: normal, \$place-items: null, \$place-content: null)</code>	<code>align-items</code> , <code>place-content</code> tiene prioridad sobre <code>justify-content</code> / <code>align-content</code> .
<code>flex-center(\$gap: null)</code>	Atajo de centrado.
<code>flex-gap(\$value: 1rem, \$var-name: --flex-gap)</code>	Establece una propiedad personalizada y <code>gap: var(--flex-gap)</code> .
<code>flex-grow(\$grow: 1)</code>	<code>flex-grow</code> .
<code>flex-shrink(\$shrink: 1)</code>	<code>flex-shrink</code> .
<code>flex-basis(\$basis: auto)</code>	<code>flex-basis</code> .
<code>flex-item(\$grow: 1, \$shrink: 1, \$basis: auto)</code>	<code>flex: <grow> <shrink> <basis></code> .
<code>flex-item-center</code>	<code>align-self: center</code> .
<code>flex-item-full</code>	<code>flex: 1 1 100%</code> .
<code>.toolbar { @include m.flex-container(row, nowrap, 0.5rem, center, space-between); }</code>	
<code>.logo { @include m.flex-item(0, 0, auto); }</code>	

Utilidades (as u)

Fuente: `_utilities.scss`. Consolida los antiguos submodulos `partials/utils/` (`maps`, `spacing`, `sizing`, `flex`, `effects`, `layout`, `core`) en un solo modulo.

Mapas de tokens de utilidad (todos !default)

Los mapas de `sizing`, `spacing`, `gap`, `font-size` y `border-width` viven en `_utilities.scss`. Los mapas restantes (`$shadow-map`, `$radius-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$font-weight-values`, `$text-align-values`, `$display-values`, `$position-values`, `$overflow-values`, `$white-space-list`, `$overflow-wrap-list`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`) viven en `_mixins.scss`, donde se comparten con el registro `@apply`.

\$sizing-map — valores de `width`/`height`/`min`/`max`:

0 0 . base 1px . 05 0.125rem . 1 0.25rem . 2 0.5rem . 3 0.75rem . 4 1rem . 5 1.25rem . 6 1.5rem . 8 2rem . 10 2.5rem . 12 3rem . 16 4rem . 20 5rem . 24 6rem . 32 8rem . 40 10rem . 48 12rem . 64 16rem . auto auto . full 100% . screen 100vw . min min-content . max max-content . fit fit-content.

\$spacing-map — valores de padding/margin/gap (compartido por el generador de clases y el registro @apply):

0 0 . 05 0.125rem . base 1px . 1 0.25rem . 1-5 0.375rem . 2 0.5rem . 2-5 0.625rem . 3 0.75rem . 3-5 0.875rem . 4 1rem . 5 1.25rem . 6 1.5rem . 7 1.75rem . 8 2rem . 9 2.25rem . 10 2.5rem . 11 2.75rem . 12 3rem . 14 3.5rem . 16 4rem . 20 5rem . 24 6rem . 28 7rem . 32 8rem . 36 9rem . 40 10rem . 44 11rem . 48 12rem . 52 13rem . 56 14rem . 60 15rem . 64 16rem . 72 18rem . 80 20rem . 96 24rem.

\$font-size-map — xs 0.75rem . sm 0.875rem . base 1rem . lg 1.125rem . xl 1.25rem . 2xl 1.5rem . 3xl 1.875rem . 4xl 2.25rem. Emitido automaticamente como .text-xstext-4xl.

\$border-width-map — 0 0 . 2 2px . 4 4px . 8 8px. Emitido automaticamente como .border-0/.border-2/.border-4/.border-8; .border (1px solid) tambien se emite.

\$flex-direction-map — row, row-reverse, col, col-reverse. **\$flex-wrap-list** — wrap, nowrap. **\$align-items-map** — start (flex-start), center, end (flex-end), stretch. **\$justify-content-map** — start, center, end, between (space-between), around (space-around), evenly (space-evenly).

\$shadow-map (efectos) — none, sm, md, lg, xl, 2xl, inner. **\$radius-map** — none 0 . sm 0.125rem . md 0.375rem . lg 0.5rem . xl 0.75rem . 2xl 1rem . 3xl 1.5rem . full 9999px. **\$z-layers-map** — auto, 0, 10, 20, 30, 40, 50, dropdown, sticky, fixed, modal, popover, tooltip. **\$transition-duration-map** — 75, 100, 150, 200, 300, 500, 700, 1000 (ms). **\$transition-timing-map** — linear, in, out, in-out. **\$opacity-values** — 0, 5, 10, 20, 25, 30, 40, 50, 60, 70, 75, 80, 90, 95, 100.

\$font-weight-values — thin 100 . extralight 200 . light 300 . normal 400 . medium 500 . semibold 600 . bold 700 . extrabold 800 . black 900. **\$text-align-values** — left, center, right, justify.

\$display-values (un mapa para que hidden pueda mapear a none) — block, inline-block, inline, flex, inline-flex, grid, inline-grid, table, table-row, table-cell, hidden (none), contents. **\$position-values** — static, fixed, absolute, relative, sticky. **\$overflow-values** — auto, hidden, scroll, visible. **\$white-space-list** — nowrap, pre, normal. **\$overflow-wrap-list** — break-word.

Mixins nucleo — *_mixins.scss*

Mixin	Comportamiento
<code>vars-list(\$list, \$prefix: null)</code>	Emite <code>--{prefix}-{value}: {value}</code> (o <code>--{value}</code>) para cada elemento de una lista.
<code>vars-map(\$map, \$prefix: null)</code>	Emite <code>--{prefix}-{key}: {value}</code> (o <code>--{key}</code>) para cada par de un mapa.
<code>absolute-center(\$pos: absolute, \$y: 50%, \$x: 50%)</code>	<code>position: top: \$y, left: \$x, transform: translate(-50%, -50%).</code>
<code>center-mx(\$xvalue: auto)</code>	<code>margin-inline: \$xvalue.</code>
<code>center-my(\$yvalue: auto)</code>	<code>margin-block: \$yvalue.</code>
<code>utils-classes(\$input, \$property, \$prefix: null)</code>	Genera <code>.prefix-value { property: value }</code> (o sin prefijo) a partir de una lista o un mapa. El sufijo de clase es la <i>clave</i> para un mapa, el <i>valor</i> para una lista.
<code>utils-classes-hover(\$input, \$property, \$prefix: null)</code>	Igual, pero emite <code>.prefix-key: hover.</code>
<pre>@include m.utils-classes((auto, hidden), overflow, "overflow"); // .overflow-auto, .overflow-hidden</pre>	
<pre>@include m.utils-classes((auto: auto, hidden: hidden), overflow, "overflow"); // salida identica</pre>	
<pre>@include m.utils-classes((flex, grid), display); // .flex, .grid (sin prefijo)</pre>	

Generadores de clases (opt-in, invocados por *generate-all-utilities()*)

Generador	Clases
<code>u.get-sizing-classes(\$map: u.\$sizing-map)</code>	<code>.w-*, .min-w-*, .max-w-*, .h-*, .min-h-*, .max-h-*</code> .
<code>u.generate-flex-utilities(\$direction: ..., \$wrap: ..., \$align: ..., \$justify: ..., \$gap: ...)</code>	<code>.flex-row / .flex-col / reversas, .flex-wrap / .flex-nowrap, .items-*, .justify-*, .gap-0gap-8.</code>
<code>u.generate-effects-utilities()</code>	<code>.shadow-*, .rounded-*, .z-*, .duration-*, .ease-*, .opacity-*</code> .

Generador	Clases
<code>u.generate-layout-utilities()</code>	<code>display</code> , <code>position</code> , <code>.overflow-*</code> , <code>.text-left/center/right/justify</code> , <code>.font-*</code> , <code>.whitespace-*</code> , <code>.wrap-break-word</code> .
<code>u.generate-all-utilities()</code>	Sizing + flex + efectos + layout.

Las clases de espaciado, tamaño de texto y ancho de borde **no** forman parte de estos generadores; se emiten automáticamente cuando se carga el módulo (ver abajo).

Salida automática cuando se carga el módulo

Cargar `utilities` ejecuta las reglas automáticas en el nivel superior. La hoja compilada por lo tanto siempre contiene:

- **Espaciado** — para cada clave de `$spacing-map`: `.p-*`, `.px-*`, `.py-*`, `.pt-*`, `.pr-*`, `.pb-*`, `.pl-*`, `.m-*`, `.mx-*`, `.my-*`, `.mt-*`, `.mr-*`, `.mb-*`, `.ml-*`, `.gap-*`, `.gap-x-*`, `.gap-y-*`.
- **Tipografía** — `.text-xs ... .text-4xl` de `$font-size-map`.
- **Anchos de borde** — `.border-0`, `.border-2`, `.border-4`, `.border-8` de `$border-width-map`, mas `.border { border-width: 1px; border-style: solid }`.
- **Efectos base** — `.gap { gap: 1rem; }`, `.shadow` (sombra por defecto) y `.rounded { border-radius: 0.25rem; }`.

```
.px-4 { padding-left: 1rem; padding-right: 1rem; }
.my-8 { margin-top: 2rem; margin-bottom: 2rem; }
.gap-x-4 { column-gap: 1rem; }
.text-lg { font-size: 1.125rem; }
.border-2 { border-width: 2px; }
.border { border-width: 1px; border-style: solid; }
```

Todo lo demás es opt-in via los generadores anteriores (o `main.scss`).

Notas sobre nombres

- Las claves son strings literales, mantenidas en kebab-case: `.p-05`, `.p-1-5`, `.p-2-5`, `.p-3-5`, `.p-base`, `.w-screen { width: 100vw }`, `.hidden { display: none }`, `.font-normal { font-weight: 400 }`.
- `generate-layout-utilities` emite `.hidden` porque `$display-values` mapea `hidden` -> `none`.

El sistema apply (as m)

Fuente: `_mixins.scss` (fusionado del antiguo `_apply.scss`).

```
@mixin register-utility($name, $styles)
```

Registra una utilidad con nombre. `$styles` es un mapa de declaraciones CSS.

```
@mixin apply($utilities...)
```

Emite cada declaracion registrada para cada nombre de utilidad, en orden. Lanza un `@error` nombrando el nombre problematico cuando una utilidad no esta registrada.

```
@use "katanakit-css/src/scss/mixins" as m;
```

```
@include m.register-utility("fade-in", (animation: fade-in 300ms  
ease));  
.card {  
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-  
white);  
}
```

Registro automatico

`_mixins.scss` registra un gran conjunto de utilidades al cargarse. La mayoría del registro se deriva de los **mismos mapas** que impulsan los generadores de clases (`$radius-map`, `$shadow-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$display-values`, `$position-values`, `$overflow-values`, `$text-align-values`, `$font-weight-values`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`), por lo que los nombres coinciden exactamente con las claves de los mapas (`p-4`, `m-2`, `gap-4`, `rounded-lg`, `shadow-md`, `z-10`, `duration-200`, `ease-out`, `opacity-50`, ...). Los nombres de espaciado siguen el mapa de tokens `$spacing-scale` de `_variables.scss`; el conjunto de clases de `$spacing-map` es mas amplio que el conjunto registrado.

Categorías:

- **Display** — `block`, `inline-block`, `inline`, `flex`, `inline-flex`, `grid`, `hidden`, mas cada clave de `$display-values`.
- **Flexbox** — `flex-row`, `flex-col`, `flex-row-reverse`, `flex-col-reverse`, `flex-wrap`, `flex-nowrap`, `flex-1`, `flex-auto`, `flex-none`, `items-*`, `justify-*`.

- **Espaciado** — para cada clave de `$spacing-scale` (variables): `p-*`, `px-*`, `py-*`, `pt-*`, `pb-*`, `pl-*`, `pr-*`, `m-*`, `mx-*`, `my-*`, `mt-*`, `mb-*`, `ml-*`, `mr-*`, `gap-*`, `gap-x-*`, `gap-y-*`.
- **Tipografía** — `text-xs (0.75rem) ... text-4xl (2.25rem)`, `text-left`, `text-center`, `text-right`, `font-thin ... font-black`.
- **Dimensiones** — `w-full`, `w-screen`, `w-auto`, `h-full`, `h-screen`, `h-auto`.
- **Bordes** — `rounded`, `rounded-md`, `rounded-lg`, `rounded-xl`, `rounded-full`, `border`, `border-0`, `border-2`, mas cada clave de `$radius-map` (`rounded-none ... rounded-full`).
- **Efectos** — `shadow`, `shadow-sm ... shadow-2xl` (cada clave de `$shadow-map`), `opacity-0 ... opacity-100` (cada clave de `$opacity-values`), `z-*`, `duration-*`, `ease-*`.
- **Helpers de grid** — `grid-cols-1`, `grid-cols-2`, `grid-cols-3`, `grid-cols-4`, `grid-cols-6`, `grid-cols-12`, `col-span-full`.
- **Posicion/overflow** — `static`, `fixed`, `absolute`, `relative`, `sticky`, `overflow-auto/hidden/scroll/visible`.
- **Colores** — `text-white`, `text-black`, `bg-white`, `bg-black`, `bg-transparent`.
- **Transiciones** — `transition`, `transition-colors`, `transition-opacity`, `transition-transform`.
- **Espacio en blanco / envoltura** — `whitespace-normal/pre/normal`, `wrap-break-word`.

Importante. Registrar una utilidad para `apply` **no** crea una clase CSS. Como el espaciado, los tamanos de texto y los anchos de borde ahora se generan como clases automaticamente, los nombres exclusivos del registro se limitan a `flex-1`, `flex-auto`, `flex-none`, `grid-cols-1/2/3/4/6/12`, `col-span-full` y los helpers `transition*`; esos nombres solo existen dentro de `apply()` y nunca deben asumirse como clases en el CSS compilado.

Manejo de errores

Claves invalidas, tipos de argumentos incorrectos y direcciones no soportadas lanzan mensajes `@error` en tiempo de compilacion que nombran el valor problematico y, cuando corresponde, las opciones validas. No hay fallback silencioso.

Arquitectura

Este documento explica como esta organizado katanakit-css, como se relacionan los modulos entre si y como un archivo fuente .scss se convierte en los artefactos que distribuyes.

Resumen — katanakit-css es **SCSS de tiempo de compilacion**. No hay runtime de JavaScript. Todo lo produce el compilador Sass a partir de mapas de tokens con !default.

1. Vision general

El framework es una coleccion de **modulos** Sass (parciales) mas dos entradas compilables:

Entrada	Que emite	Usado para
src/scss/main.scss	“Hoja completa” publica: reset + propiedades personalizadas de tokens + variables de color y utilidades + todos los generadores de utilidades. Sin componentes.	Artefacto npm dist/css/katanakit.css y consumidores de @use "katanakit-css/src/scss/main".
src/scss/demo.scss	@use "main" + @use "components/index" (componentes de ejemplo contruidos sobre apply).	Demo local solamente (Vite).

Los componentes se mantienen intencionalmente fuera de main.scss: son estilos de *ejemplo*, no API del framework.

2. Modelo de capas

```
+-----+
| PUNTOS DE ENTRADA                                     |
|   main.scss   .   demo.scss                           |
+-----+
```

```

| @use
+-----v-----+
| MODULOS NUCLEO (src/scss/*.scss)
|   reset . variables(v) . functions(f) . mixins(m)
|   utilities(u)
+-----+-----+
| @use/@forward      | @use
+-----v-----+ +-----v-----+
| (fusionado en _mixins.scss) | | COMPONENTES (solo demo)
| breakpoints . grid . flex   | |   components/_index.scss
| apply . theme . core        | |   usa mixins
+-----+-----+ +-----+-----+

```

Los modulos nunca se @import entre si; usan @use moderno (minusculas) con namespaces explicitos. Las anteriores capas partials/ y partials/utils/ han sido consolidadas: _variables.scss absorbe colores y tema, _mixins.scss absorbe breakpoints, grid, flex, apply y generadores nucleo, y _utilities.scss absorbe todos los mapas y generadores de clases.

3. Responsabilidades de cada parcial

Parcial	Namespace	Responsabilidad
_reset.scss	—	Reset moderno. Los valores por defecto se exponen como propiedades personalizadas :root (--m-reset, --box-sizing, --min-h-screen, ...) para que un tema pueda cambiarlos sin editar el reset.
_variables.scss	as v	Mapas de tokens ! default (fuentes, sombras, contenedores, breakpoints, espaciado, radios, z, duraciones, temporizaciones), funciones de acceso,

Parcial	Namespace	Responsabilidad
		generate-css-tokens(), tokens/funciones de color (get-color/get/alpha), generate-css-vars(), utilidades de color y theme().
_functions.scss	as f	Helpers puros: conversion de unidades, fluid() -> clamp(), manipulacion de color, color-mix-var(), to-class(). Sin salida CSS.
_mixins.scss	as m	Consultas de breakpoint (breakpoint()/bp()), alias con nombre, consultas de características), mixins de composicion de grid, mixins de contenedor/elemento flex, registro @apply (register-utility()/apply()), generadores nucleo (utils-classes, vars-list/vars-map, helpers de centrado) y hook de tema.
_utilities.scss	as u	Mapas de tokens de utilidad (\$sizing-map, \$font-size-map, \$border-width-map, etc.) mas todos los generadores de clases (get-sizing-classes, generate-flex-utilities, generate-

Parcial	Namespace	Responsabilidad
		effects-utilities, generate-layout- utilities, generate- all-utilities).

4. El motor de utilidades

El sistema de utilidades esta deliberadamente impulsado por mapas para que los tokens y las clases generadas no puedan desviarse.

4.1 Los mapas son la fuente unica de verdad

`_utilities.scss`, `_variables.scss` y `_mixins.scss` contienen los mapas de tokens de utilidad. Los mapas de `sizing`/`font-size`/`border-width` viven en `_utilities.scss`; `$spacing-map` vive en `_variables.scss` (compartido por el generador de clases y el registro `@apply`); el resto vive en `_mixins.scss` junto al generador de clases generico:

- `$sizing-map` (valores de width/height, incl. full, screen, min, max, fit),
- `$spacing-map` (escala de padding/margin/gap; las claves son literales: "05", "1-5", "2-5", "3-5", "base" ...),
- `$font-size-map` (.text-xstext-4xl), `$border-width-map` (.border-0/2/4/8),
- `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map`,
- `$shadow-map`, `$radius-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`,
- `$font-weight-values`, `$text-align-values`, `$display-values`, `$position-values`, `$overflow-values`, `$white-space-list`, `$overflow-wrap-list`.

Todos los mapas son !default, por lo que los proyectos dependientes pueden sobreescribirlos con la clausula `with` de Sass.

4.2 De mapas a clases CSS

```

_utilities.scss (mapas) -> generadores (_utilities.scss)
    |  usa m.utils-classes($input,
$property, $prefix)
    |
    v
    .w-full { width: 100% }
    .hidden { display: none }    // $display-values

```

mapea hidden -> none

```
.font-normal { font-weight: 400 }  
.gap-4 { gap: 1rem }  
...
```

- `_mixins.scss` proporciona los mixins genericos `utils-classes()` / `utils-classes-hover()` mas helpers de variables CSS (`vars-list`, `vars-map`) y helpers de centrado (`absolute-center`, `center-mx`, `center-my`).
- `_utilities.scss` emite, automaticamente al cargarse: clases de padding y margin en todas las direcciones (`.p-*`, `.px-*`, `.py-*`, `.pt-*`, ..., `.m-*`, `.mx-*`, `.my-*`, ...), `gap-*/gap-x-*/gap-y-*` para cada clave de `$spacing-map`, tamanos de texto (`.text-xs` ... `.text-4xl`), anchos de borde (`.border`, `.border-0/2/4/8`), mas `.gap`, `.shadow` y `.rounded`. Esto significa que cargar el modulo es suficiente para obtener las clases comunes de espaciado y tipografia.
- El resto es **opt-in**: `get-sizing-classes()`, `generate-flex-utilities()`, `generate-effects-utilities()` y `generate-layout-utilities()`. `generate-all-utilities()` las llama a todas, lo que invoca `main.scss`.

4.3 El registro *apply* refleja los mismos mapas

`_mixins.scss` construye su registro automatico con bucles `@each` sobre **los mismos mapas de utilidad que impulsan los generadores de clases** — `$radius-map`, `$shadow-map`, `$z-layers-map`, `$transition-duration-map`, `$transition-timing-map`, `$opacity-values`, `$display-values`, `$position-values`, `$overflow-values`, `$text-align-values`, `$font-weight-values`, `$flex-direction-map`, `$flex-wrap-list`, `$align-items-map`, `$justify-content-map` — mas el mapa de tokens `$spacing-scale` de `_variables.scss` para los nombres de espaciado**. Esto mantiene `apply(flex, p-4, rounded-lg, ...)` consistente con las clases generadas `.flex`, `.p-4` y `.rounded-lg`, y significa que un cambio de escala se propaga a ambos sistemas a la vez.

El registro tambien lleva helpers escritos a mano que *no* se generan como clases (`flex-1`, `flex-auto`, `flex-none`, columnas de grid (`grid-cols-*`, `col-span-full`), transiciones, ...). Esos nombres solo estan disponibles dentro de `@include a.apply(...)`.

4.4 Utilidades de color

Las utilidades de color **no** pasan por el motor generico. `_variables.scss` las emite con un helper dedicado que itera `$colors` x `$shades` y `$special-colors`:

```
.text-neutral-500 { color: hsl(0 0% 35%) }
.bg-info-300      { background-color: ... }
.border-white     { border-color: ... }
.hover-bg-success-600:hover { ... }
```

Las propiedades personalizadas de paleta (--neutral-500, --white, ...) son una salida paralela de `generate-css-vars()`; las clases de utilidad mantienen **valores literales** para que la purga de CSS basada en clases pueda eliminarlas de forma segura mientras las variables de tokens permanecen en `:root`.

5. Tokens y el bloque `:root`

`main.scss` llama, en orden:

```
@use "./reset";           // el reset emite sus propios :root
defaults
@use "./variables" as v;
@use "./functions" as f;
@use "./mixins" as m;
@use "./utilities" as u;

@include v.generate-css-tokens(); // --font-*, --shadow-*, --spacing-
*, ...
@include v.generate-css-vars();   // --neutral-100..., --white, ...
@include v.all-utilities();       // .text-*, .bg-*, .border-*, hover
@include u.generate-all-utilities();
```

Las propiedades personalizadas CSS se agrupan bajo `:root`. Como cada mapa de tokens es `!default`, un consumidor puede reconfigurar toda la salida cargando el modulo variables con una clausula `with` *antes* de que cualquier otro lo cargue (un modulo Sass solo puede configurarse una vez por compilacion).

Los temas oscuros no editan `:root`: `theme("dark")` emite un bloque *adicional* bajo `:root[data-theme="dark"]` con cada paleta invertida (100 ↔ 700, 200 ↔ 600, 300 ↔ 500), dejando los valores por defecto claros intactos.

6. Canalizacion de compilacion

Dos canalizaciones independientes producen los artefactos:

```
(1) artefacto npm
    src/scss/main.scss
      | yarn build:css
      | sass src/scss/main.scss dist/css/katanakit.css
      | --no-source-map --style=compressed
      v
    dist/css/katanakit.css      <-- distribuido, referenciado
por "style"
```

```
(2) demo local
    src/scss/demo.scss + index.html + demo/main.js
      | yarn dev          -> vite dev server (HMR, puerto 4321)
      | yarn build:demo -> vite build
      |                               PostCSS autoprefixer + PurgeCSS
      v
    demo-dist/ (index.html + assets/*.css + assets/*.js)
```

Notas sobre la compilación de la demo (vite.config.js):

- `publicDir: false` — no hay carpeta estática; el CSS se compila desde el grafo de módulos SCSS.
- El contenido de PurgeCSS es `index.html + demo/**/*.{js,ts}`. Una lista segura mantiene tokens que parecen variantes (`hover:*`, `md:*`, ...) y se preservan keyframes/font-faces.
- `variables: false` — PurgeCSS mantiene las propiedades personalizadas en `:root` (las variables de tokens/color) aunque las clases de utilidad usen valores literales.
- La salida va a `demo-dist/`, intencionalmente **no** a `dist/`, para que la demo nunca sobrescriba el artefacto CSS de npm.
- El artefacto `dist/css/katanakit.css` lo produce el CLI de Sass solo; ejecuta tu propio PostCSS (por ejemplo autoprefixer) cuando consumes el fuente SCSS si necesitas prefijos para navegadores antiguos.

La suite de pruebas (vitest, `test/**/*.test.mjs`) compila las entradas y fixtures en memoria via la API JS de Sass (`loadPaths: src/scss`) y afirma sobre el CSS compilado; nunca escribe en `src/` ni `dist/`.

7. Convenciones que mantienen la arquitectura intacta

1. **Una responsabilidad por parcial** — pon nuevas capacidades en el módulo que posee esa preocupación (tokens y colores en `_variables.scss`, mixins

en `_mixins.scss`, mapas de utilidad y generadores en `_utilities.scss`, ...).

2. **Solo sintaxis de modulos moderna** — `@use/@forward`, nunca `@import`.
3. **Los mapas son !default** — cualquier escala puede sobrescribirse sin forks.
4. **Claves literales en kebab-case** — las claves de utilidad (`.p-05`, `.p-1-5`, `.w-screen`) coinciden exactamente con las claves del mapa; no hay capa de conversion de corchetes.
5. **Clases generadas vs registro** — solo los mixins emiten clases explicitamente; registrar una utilidad para `apply` nunca crea una clase por si sola.
6. **main.scss permanece sin componentes** — los componentes de ejemplo viven en `components/_index.scss` y los `demo.scss`, nunca la hoja publica.

Ejemplos

Cuatro ejemplos funcionales viven en el directorio `examples/` del repositorio. Cada uno es un archivo SCSS independiente que puedes compilar con:

```
sass --load-path src/scss examples/<categoria>/<archivo>.scss out.css
```

1. Tokens de diseno personalizados

`examples/tokens/custom-tokens.scss` — sobrescribe los mapas de tokens !default con una clausula `with` **antes** de que cualquier cosa los use, luego consume los nuevos valores a traves de los accesadores.

```
// =====  
//  examples/tokens/custom-tokens.scss  
//  Cómo sobrescribir los tokens del framework ANTES de usarlos.  
//  =====
```

```
// 1) Override de mapas con @use ... with (antes de cualquier @include)
```

```
@use "variables" as v with (  
  $spacing-scale: (  
    "0": 0,  
    "1": 0.5rem,  
    "2": 1rem,  
    "3": 1.5rem,  
    "4": 2rem
```

```

    ),
    $radius: (
      "default": 0.5rem,
      "full": 9999px
    )
  );

// 2) Emite los tokens como custom properties en :root
@include v.generate-css-tokens();

// 3) Usa las funciones de acceso con los nuevos valores
.hero {
  padding: v.spacing(4);
  border-radius: v.radius("default");
  box-shadow: v.shadow("lg");
  font-family: v.font-family("sans-serif");
}

```

La idea clave: **configurar una vez, antes del primer uso**. Como un modulo Sass solo puede configurarse una vez por compilacion, la clausula with debe venir antes de que cualquier otro modulo cargue variables. Consulta Tokens de diseno para la referencia completa de tokens.

2. Layout de dashboard con grid

examples/layout/grid-dashboard.scss — un dashboard construido con los mixins de grid: un contenedor de 12 columnas, spans explicitos, un colapso mobile-first via md-down y una rejilla responsiva de tarjetas con auto-fill.

```

// =====
//  examples/layout/grid-dashboard.scss
//  Layout de dashboard con los mixins de grid y breakpoints.
//  =====

@use "mixins" as m;
@use "variables" as v;

.dashboard {
  @include m.grid-container(12, $gap: 1rem, $place-items: stretch,
    $place-content: stretch);

  // Sidebar: ocupa 3 columnas, el contenido 9
  .sidebar {

```

```

    @include m.grid-span(3);
  }

  .content {
    @include m.grid-span(9);
  }

  // En pantallas medianas o menores, todo apila a 1 columna
  @include m.md-down {
    grid-template-columns: 1fr;

    .sidebar,
    .content {
      @include m.grid-span(1);
    }
  }
}

// Rejilla responsive de tarjetas con auto-fill
.cards {
  @include m.grid-responsive(280px, fill, 1fr, 1.5rem);
}

// Colocación explícita de un ítem
.pinned-card {
  @include m.grid-placement(2, 4, 1, 2);
}

// Centrado absoluto
.modal-overlay {
  @include m.grid-center(0);

  .modal {
    @include m.grid-item-center;
    max-width: v.container("md");
  }
}

```

El sidebar y el contenido comparten la misma grid de 12 columnas; por debajo de md todo vuelve a una sola columna. Consulta los mixins de Grid para cada mixin usado aquí.

3. Tema oscuro

examples/theme/dark-mode.scss — publica las paletas invertidas bajo :root[data-theme="dark"] y añade un hook de preferencia del sistema encima.

```
// =====
//  examples/theme/dark-mode.scss
//  Tema oscuro automático: las paletas se invierten (tono 100
//  ↔ 700) bajo :root[data-theme='dark'].
//  =====

@use "variables" as v;
@use "mixins" as m;

// Emite :root[data-theme='dark'] con las paletas invertidas
@include v.theme("dark");

// Y añade tu propio hook de sistema si quieres:
@include m.dark-mode {
  :root {
    color-scheme: dark;
  }
}
```

Luego cambia el atributo desde el markup o JavaScript:

```
<html data-theme="dark">...</html>
```

Lee Modo oscuro para la tabla de inversion y los fragmentos de JavaScript.

4. Componentes de boton y tarjeta

examples/components/button-card.scss — componentes compuestos construidos con el sistema @apply.

```
// =====
//  examples/components/button-card.scss
//  Componentes compuestos con el sistema @apply propio.
//  =====

@use "mixins" as m;

.card {
  @include m.apply(flex, flex-col, p-4, rounded-lg, shadow-md, bg-
```

```

white);

    &:hover {
        @include m.apply(shadow-lg);
    }
}

.button {
    @include m.apply(
        inline-flex,
        items-center,
        justify-center,
        px-4,
        py-2,
        rounded-md,
        font-semibold,
        transition-colors
    );

    &:hover {
        @include m.apply(bg-white, text-black);
    }
}

```

Cada componente permanece como un unico bloque legible de nombres de utilidad; el registro los expande a declaraciones puras en tiempo de compilacion, por lo que el CSS de salida no contiene clases de utilidad para estos componentes.

Hoja de ruta

Una lista viva de hacia donde se dirige `katanakit-css` (mini-framework SCSS). Las contribuciones son bienvenidas; elige un elemento, discutelo en un issue y abre un pull request.

Leyenda: [x] hecho . [] planificado (sin compromiso de orden).

Estado actual

El framework esta en **0.1.0 (no publicado)**. El ambito enviado esta documentado en [CHANGELOG.md](#); la API en la Referencia API.

Enviado para 0.1.0

- ☒ Arquitectura SCSS modular con parciales modernos @use/@forward y una entrada publica sin componentes (src/scss/main.scss).
- ☒ Sistema de tokens de diseno — mapas !default para fuentes, sombras, contenedores, breakpoints, espaciado, radios, z-index, duraciones de transicion y temporizaciones, emitidos como propiedades personalizadas CSS (--font-*, --shadow-*, --spacing-*, --radius-*, --z-*, --duration-*, --ease-*, ...).
- ☒ Sistema de color — seis paletas semanticas (neutral, purple, info, warning, danger, success) con **7 tonos cada una** (100–700) mas cuatro colores especiales; variables CSS, clases de utilidad (text/bg/border/ hover) e inversion de tema oscuro via :root[data-theme="dark"].
- ☒ Breakpoints responsivos — 7 niveles (xs ... 3xl), generico breakpoint()/bp() con up/down/only/between, alias up y down con nombre (xxl/xxxl, xxl-down/xxxl-down) y consultas de características.
- ☒ Mixins CSS Grid — auto-fill/fit responsivo, columnas fijas, columnas por breakpoint, contenedores, areas, colocacion, apilamiento, helpers de elementos, subgrid y (experimental) masonry.
- ☒ Mixins Flexbox — contenedor, centrado, gap y helpers de elementos.
- ☒ Funciones puras — rem(), px(), to-unit(), strip-unit(), fluid() -> clamp(), tint(), shade(), saturate-color(), desaturate-color(), complement(), contrast(), color-mix-var(), to-class().
- ☒ Motor de utilidades impulsado por mapas de _utilities.scss — clases de padding y margin en todas las direcciones (.p-*, .px-*, .py-*, .pt-*, .m-*, .mx-*, .my-*, ...), gap-*/gap-x-*/gap-y-*, tamanos de texto (.text-xstext-4xl) y anchos de borde (.border, .border-0/2/4/8) activos por defecto; generadores de sizing/flex/effects/layout opt-in.
- ☒ Registro al estilo @apply (register-utility + apply) mantenido en sincronia con los mapas de utilidad.
- ☒ Reset preflight extendido al estilo Tailwind cuyos valores por defecto son propiedades personalizadas sobreescribibles.
- ☒ Demo con Vite (index.html + demo/main.js) con HMR, un **selector de version** (demo/version-switcher.js, yarn build:versions -> public/versions/) y una compilacion de produccion con PurgeCSS en demo-dist/.
- ☒ Componentes de ejemplo contruidos sobre @apply (components/_index.scss).

- ☒ Suite de pruebas (vitest, 26 pruebas sobre fixtures) y documentacion completa en ingles (README + docs/ + CONTRIBUTING/SECURITY/CHANGELOG), mas el sitio de documentacion Astro + Starlight en site/.

Planificado

Utilidades como clases reales

- ❑ Extraer la generacion de clases de gap/margin/padding direccionales en un bucle dedicado para que futuras escalas puedan anadir claves sin tocar el bloque de auto-emision.

Arquitectura de tokens

- ❑ **Unificar las dos escalas de espaciado:** \$spacing-scale (variables de tokens y el registro @apply) y \$spacing-map (clases de utilidad) actualmente contienen conjuntos de claves diferentes. Unificarlas haria que las variables --spacing-*, las clases .p-* y los nombres de apply() fueran una sola fuente de verdad.
- ❑ Prefijo configurable para las propiedades personalizadas CSS generadas (un \$prefix opcional en generate-css-tokens()/generate-css-vars()).
- ❑ Evaluar un tercer nivel de escala (tamanos 3x1/4x1) y alias de opacidad/grosor con nombre.

Colores

- ❑ Extender el sistema de paletas mas alla de 7 tonos (por ejemplo tonos 50 y 800/900) sin romper los consumidores existentes de 100–700.
- ❑ Variantes hover para border-* (hover-border-*), reflejando hover-text-*/hover-bg-*.
- ❑ Documentacion para la paleta de marca personal (colors-palette.md) como tema de partida, mantenido separado de las paletas del framework.

Componentes y extras

- ❑ Distribuir un modulo components opt-in en npm (solo los mixins/registro que necesita), manteniendo la hoja publica sin componentes.
- ❑ Mixins de ayuda contenedor/max-w sobre los tokens --container-*.

DX y herramientas

- ❑ Integracion continua con GitHub Actions (probar con multiples versiones de Dart Sass, compilar ambos artefactos, cachear Yarn).
 - ❑ Documentar la biblioteca TypeScript hermana katanakit-js junto a este repositorio para que los dos proyectos esten claramente distinguidos.
-

Contribuir a la hoja de ruta

Si quieres abordar un elemento planificado, abre un issue primero para acordar el enfoque (algunos elementos toman decisiones de API que rompen compatibilidad). Consulta [CONTRIBUTING.md](#) para el flujo de desarrollo.